



UNIMORE

UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche, Informatiche e Matematiche

Corso di Laurea in Informatica

Controllo semaforico adattivo mediante strategie Max-Pressure: progettazione e valutazione tramite simulazione

Relatore:

Prof. Giacomo Cabri

Candidato:

Jakub Pawel Homoncik

Anno Accademico 2025–2026

Sommario

Il controllo semaforico a tempi fissi non utilizza informazioni sul traffico osservato e può quindi assegnare il verde a direzioni poco richieste mentre si formano code su altri accessi. Questa tesi studia strategie di controllo adattivo basate sul principio Max-Pressure, nel quale ogni incrocio seleziona la fase da servire confrontando le code a monte e a valle dei movimenti consentiti. Il lavoro comprende una piattaforma sperimentale riproducibile basata su SUMO e TraCI, un controller Max-Pressure con sette estensioni specialistiche e una pipeline per analisi di ablazione, esecuzioni parallele e produzione automatica di metriche e report.

La prima valutazione completa riguarda una rete sintetica Manhattan 8×8 , tre livelli di domanda e cinque seed. I 135 casi sperimentali sono terminati senza errori e con censura nulla. Rispetto al programma semaforico statico nativo, il miglior controller riduce il tempo medio di attesa del 64,26% nel traffico leggero, del 60,09% nel traffico medio e del 53,05% nel traffico elevato. I risultati mostrano inoltre che l'aggiunta di una regola specialistica non produce necessariamente un beneficio: alcune estensioni migliorano soltanto in specifici regimi, altre rimangono inattive oppure introducono un costo superiore al vantaggio ottenuto.

La seconda valutazione riguarda la rete reale di Bologna, con tre distribuzioni della domanda, tre livelli di traffico e tre seed. In questo scenario il Max-Pressure puro risulta sempre peggiore del programma semaforico nativo. Sono stati quindi sviluppati due controller ibridi che conservano il piano esistente nei regimi appropriati e attivano Max-Pressure in base al carico osservato. Nelle 270 simulazioni considerate, completate senza censura, è sempre uno dei due ibridi a ottenere la minore attesa media: `mp_super` prevale in cinque gruppi e `mp_program0` in quattro. Il risultato mostra che, su una rete reale, l'adattività può essere più efficace quando integra il piano nativo anziché sostituirlo continuamente.

Indice

Sommario	i
1 Introduzione	1
1.1 Contesto e motivazioni	1
1.2 Problema affrontato	2
1.3 Obiettivi del lavoro di tesi	2
1.4 Contributi del lavoro	3
1.5 Organizzazione della tesi	3
2 Contesto e stato dell'arte	4
2.1 Controllo semaforico urbano	4
2.1.1 Controllo a tempi fissi	4
2.1.2 Controllo attuato e adattivo	4
2.2 Strategie Max-Pressure	5
2.2.1 Principio di funzionamento	5
2.2.2 Pressione dei movimenti e delle fasi	6
2.2.3 Proprietà e limiti del controllo locale	6
2.3 Estensioni del Max-Pressure	6
2.3.1 Costo delle transizioni semaforiche	6
2.3.2 Prevenzione dello spillback	7
2.3.3 Fairness e prevenzione della starvation	7
2.3.4 Gestione dei plotoni di veicoli	7
2.4 Simulazione microscopica del traffico	8
2.4.1 SUMO	8
2.4.2 Interfaccia TraCI	8
2.5 Situazione iniziale del progetto	8
2.5.1 Componenti software già disponibili	9
2.5.2 Limiti della soluzione iniziale	9
2.5.3 Contributi sviluppati nella tesi	9
3 Requisiti e progettazione del sistema	10
3.1 Requisiti funzionali	10
3.2 Requisiti non funzionali	10

3.2.1	Riproducibilità	10
3.2.2	Modularità	11
3.2.3	Osservabilità	11
3.3	Architettura generale	11
3.3.1	Runner delle simulazioni	12
3.3.2	Generazione delle popolazioni	13
3.3.3	Controller semaforici	13
3.3.4	Raccolta e aggregazione delle metriche	13
3.4	Flusso di esecuzione di una simulazione	14
3.5	Scelte progettuali	14
3.5.1	Separazione tra controller e simulatore	14
3.5.2	Configurazione mediante parametri	15
3.5.3	Gestione delle mappe reali e sintetiche	15
4	Implementazione dei controller Max-Pressure	16
4.1	Rappresentazione dei semafori	16
4.1.1	Identificazione delle fasi principali	16
4.1.2	Costruzione dei movimenti	17
4.1.3	Conservazione delle transizioni di sicurezza	17
4.2	Controller Max-Pressure di base	17
4.2.1	Misura delle code	17
4.2.2	Calcolo della pressione	18
4.2.3	Selezione della fase	18
4.2.4	Vincoli di minimo e massimo verde	18
4.3	Macchina a stati del controller	19
4.3.1	Fase attiva e target pendente	19
4.3.2	Gestione di giallo e all-red	19
4.3.3	Recupero dalle transizioni anomale	19
4.4	Lost-Time-Aware	20
4.5	Minimo verde dinamico Nmin	20
4.6	Fairness	21
4.7	Penalità downstream	21
4.8	Anti-spillback	22
4.9	Platoon extension	22
4.10	Margini e stabilità dello switch	22
4.11	Contatori di attivazione	23
5	Metodologia e piattaforma sperimentale	24

5.1	Obiettivi della valutazione	24
5.2	Pipeline sperimentale	24
5.2.1	Generazione dei casi sperimentali	25
5.2.2	Organizzazione delle campagne	25
5.2.3	Report automatici	25
5.3	Generazione della domanda di traffico	26
5.3.1	Livelli <i>low</i> , <i>medium</i> e <i>high</i>	26
5.3.2	Seed e riproducibilità	27
5.4	Profilo di guida <i>human_light</i>	27
5.5	Controller di confronto	27
5.5.1	<i>fixed_program0</i>	27
5.5.2	<i>fixed_tuned</i>	28
5.6	Metriche	28
5.6.1	Tempo di attesa	28
5.6.2	Tempo di viaggio e <i>time loss</i>	28
5.6.3	Velocità media	28
5.6.4	Viaggi completati e tasso di completamento	28
5.6.5	Censura	29
5.7	Criteri di confronto	29
6	Valutazione su Manhattan 8×8	30
6.1	Motivazione dello scenario sintetico	30
6.2	Topologia della rete	30
6.3	Configurazione sperimentale	31
6.4	Confronto tra Max-Pressure e controllo fisso	31
6.5	Valutazione delle singole estensioni	33
6.5.1	Effetto chiaro: <i>Nmin</i>	33
6.5.2	Varianti quasi equivalenti a <i>mp_base</i>	33
6.5.3	Varianti penalizzate dalla domanda uniforme	33
6.6	Analisi dei contatori di attivazione	34
6.7	Confronto complessivo	34
6.8	Indicazioni ricavate per lo scenario reale	35
7	Adattamento alla rete di Bologna	36
7.1	Caratteristiche della rete	36
7.2	Problemi riscontrati	37
7.2.1	Complessità della topologia	37
7.2.2	Stalli e congestione	37

7.3	Preparazione della mappa e compatibilità	38
7.4	Analisi del programma semaforico <code>program0</code>	38
7.5	Limiti del Max-Pressure puro su Bologna	38
7.5.1	Prestazioni nel traffico leggero	39
7.5.2	Assenza di coordinamento esplicito	39
7.6	Calibrazione della domanda	39
7.7	Distribuzioni di domanda per Bologna	40
7.7.1	<code>balanced</code>	40
7.7.2	<code>skewed</code>	40
7.7.3	<code>peak</code>	40
7.8	Controller ibrido <code>mp_program0</code>	40
7.8.1	Misura del carico locale	40
7.8.2	Passaggio tra <code>program0</code> e Max-Pressure	41
7.8.3	Isteresi e stabilità della selezione	41
7.9	Meta-controller <code>mp_super</code>	41
7.9.1	Motivazione	41
7.9.2	Carico (load)	42
7.9.3	Squilibrio della domanda (imbalance)	42
7.9.4	Squilibrio dell'attesa (wait imbalance)	42
7.9.5	Variabilità temporale (burstiness)	43
7.9.6	Rilevazione dei plotoni (platoon score)	43
7.9.7	Saturazione downstream	43
7.9.8	Regole di selezione delle modalità	43
7.9.9	Isteresi temporale	44
7.10	Evoluzione delle soglie di <code>mp_super</code>	44
8	Valutazione finale su Bologna	45
8.1	Protocollo sperimentale	45
8.2	Risultati aggregati	46
8.3	Completezza e variabilità	48
8.4	Comportamento delle varianti specialistiche	48
8.5	Uso delle modalità di <code>mp_super</code>	48
8.6	Discussione dei risultati	49
8.7	Limiti della valutazione	50
9	Conclusioni	51
9.1	Sintesi del lavoro	51
9.2	Risultati principali	51

9.3	Risposta agli obiettivi della tesi	52
9.4	Limiti	52
9.5	Sviluppi futuri	52
A	Configurazioni sperimentali	54
A.1	Configurazioni delle campagne	54
A.2	Output e risultati completi	54
A.3	Struttura essenziale del software	55
B	Dettagli implementativi	56
B.1	Costruzione dei movimenti dalle fasi SUMO	56
B.2	Calcolo della pressione di fase	57
B.3	Selezione delle modalità di mp_super	58
	Riferimenti bibliografici	61

1 Introduzione

La regolazione semaforica decide come distribuire tra flussi in conflitto una risorsa limitata, il diritto di attraversare un incrocio. Una scelta apparentemente locale può propagarsi lungo la rete: un verde troppo breve lascia una coda residua, un verde inutile sottrae tempo agli altri accessi e una coda che raggiunge l'incrocio precedente può impedire movimenti altrimenti non conflittuali. Per questo motivo il controllo dei semafori costituisce un problema centrale nei sistemi di trasporto intelligenti.

Questo capitolo introduce il problema affrontato, gli obiettivi e i contributi del lavoro. La trattazione rimane volutamente ad alto livello, le formulazioni matematiche, le scelte implementative e il protocollo sperimentale vengono sviluppati nei capitoli successivi.

1.1 Contesto e motivazioni

I piani semaforici a tempi fissi sono semplici, prevedibili e poco costosi da gestire. Associano a ciascuna fase una durata definita in anticipo e ripetono ciclicamente la stessa sequenza. La loro efficacia dipende tuttavia dalla corrispondenza tra il profilo di domanda usato durante la progettazione e il traffico effettivamente presente. Quando la domanda cambia nello spazio o nel tempo, il piano può continuare a servire una direzione vuota mentre su un altro accesso si accumulano veicoli.

Il controllo adattivo cerca invece di usare osservazioni correnti della rete per modificare il servizio offerto. Tra le strategie decentralizzate, il Max-Pressure è particolarmente interessante perché non richiede una previsione completa della domanda né un unico ottimizzatore centrale. Ogni intersezione confronta le code a monte e a valle dei movimenti consentiti e assegna il verde alla fase con pressione maggiore. Questa famiglia deriva dalle politiche di backpressure per reti di code [TE92], nel traffico urbano, la letteratura ha mostrato importanti proprietà di stabilità e throughput in opportune ipotesi modellistiche [Var13; Won+12; Lev23].

Il passaggio dalla teoria a una simulazione microscopica realistica introduce però costi e vincoli che il modello ideale tende a semplificare: giallo e tempo di sgombero, minimo e massimo verde, capacità finita delle corsie, spillback, arrivi a plotoni e necessità di non trascurare movimenti deboli. Ne segue che “scegliere sempre la pressione massima” non è, da solo, una specifica sufficiente per un controller utilizzabile.

1.2 Problema affrontato

Il lavoro affronta due problemi collegati. Il primo è ingegneristico: costruire una piattaforma con cui implementare, eseguire e confrontare in modo riproducibile controller semaforici diversi su reti SUMO sintetiche e reali. Il secondo è sperimentale: stabilire in quali condizioni il Max-Pressure di base o le sue estensioni migliorino effettivamente il flusso rispetto a un programma statico.

L'indicatore principale è il tempo di attesa, affiancato da tempo di viaggio, perdita di tempo, velocità, numero di viaggi completati e censura. Il lavoro non persegue obiettivi ambientali: consumi di carburante ed emissioni non sono utilizzati per selezionare o valutare i controller. In questo modo l'analisi rimane allineata alla domanda di ricerca, concentrata sull'efficienza degli incroci e del flusso veicolare.

Una difficoltà metodologica importante è evitare conclusioni ottenute da un solo seed o da una simulazione interrotta prima che tutti i veicoli completino il viaggio. Un controller può mostrare una bassa attesa media semplicemente perché i veicoli più problematici sono ancora nella rete. Per questo la piattaforma registra esplicitamente la quota di viaggi non conclusi e considera la censura una condizione necessaria per interpretare correttamente le altre metriche.

1.3 Obiettivi del lavoro di tesi

Gli obiettivi sono i seguenti:

1. implementare un controller Max-Pressure decentralizzato compatibile con le logiche semaforiche presenti nelle mappe SUMO;
2. introdurre estensioni mirate a switching loss, minimo verde dinamico, fairness, capacità downstream, spillback e arrivi a plotoni;
3. realizzare una pipeline di ablation che generi popolazioni riproducibili, esegua casi in parallelo e produca report confrontabili;
4. definire metriche capaci di distinguere un reale miglioramento da un risultato condizionato da viaggi incompleti;
5. valutare separatamente le varianti su una rete Manhattan regolare, prima di affrontare la maggiore eterogeneità della rete di Bologna;
6. ricavare dai risultati indicazioni progettuali per controller capaci di adattarsi a regimi di traffico differenti.

1.4 Contributi del lavoro

Il lavoro è partito da una base software composta dal collegamento con SUMO, da un runner, da un generatore di popolazioni e da un controller fisso, che sono stati poi integrati ed estesi con:

Nel seguito, con *campagna sperimentale* si intende un insieme organizzato di simulazioni eseguite con una stessa configurazione generale, variando in modo controllato mappa, livello di domanda, seed e scenario di controllo.

- un'implementazione completa del Max-Pressure, comprensiva di macchina a stati per preservare giallo e all-red;
- sette famiglie di estensioni parametriche e relativi contatori di attivazione;
- una rappresentazione uniforme degli scenari sperimentali e dei profili di guida;
- una pipeline batch con preflight, cache delle popolazioni, parallelismo, tracciamento del progresso e report automatici;
- il calcolo esplicito di viaggi completati e censura a partire dai file `tripinfo`;
- una campagna finale su Manhattan 8×8 composta da 135 simulazioni valide, tre livelli di domanda e cinque seed;
- l'estensione successiva della metodologia a distribuzioni spaziali non uniformi e a controller ibridi per la rete reale di Bologna.

Il contributo non consiste quindi soltanto in un algoritmo, ma in un processo sperimentale osservabile. I contatori interni consentono di verificare se una variante ha prodotto un risultato perché il suo meccanismo si è attivato o se, al contrario, è rimasta operativamente identica al Max-Pressure di base.

1.5 Organizzazione della tesi

Il capitolo 2 presenta il controllo semaforico adattivo, il Max-Pressure e gli strumenti di simulazione. Il capitolo 3 definisce requisiti e architettura della piattaforma. Il capitolo 4 descrive l'implementazione del controller e delle estensioni. Il capitolo 5 formalizza il protocollo sperimentale, le popolazioni e le metriche. Il capitolo 6 analizza la campagna Manhattan 8×8 . La parte successiva della tesi applica le indicazioni ricavate alla rete di Bologna, introduce i controller ibridi sviluppati per la mappa reale e discute i risultati conclusivi.

2 Contesto e stato dell'arte

Per collocare il contributo è utile distinguere tre livelli: le strategie di controllo già note in letteratura, gli strumenti software utilizzati e le componenti effettivamente sviluppate durante il tirocinio. Il capitolo procede dal controllo a tempi fissi al Max-Pressure, ne discute le principali estensioni e conclude descrivendo lo stato iniziale del progetto.

2.1 Controllo semaforico urbano

Un'intersezione semaforizzata è descritta da un insieme di movimenti, per esempio attraversamento rettilineo o svolta, e da un insieme di fasi. Una fase abilita contemporaneamente movimenti compatibili. Il controller deve decidere quanto a lungo mantenere la fase corrente e quando avviare la transizione verso la successiva. La qualità della decisione dipende sia dal criterio di scelta sia dai vincoli fisici: un cambio di fase richiede intervalli durante i quali la capacità utile si riduce.

2.1.1 Controllo a tempi fissi

Nel controllo a tempi fissi la sequenza, la durata delle fasi e l'eventuale offset rispetto agli altri incroci sono definiti a priori. Il principale vantaggio è la prevedibilità. Un piano ben calibrato può coordinare corridoi e creare onde verdi; inoltre non dipende dall'affidabilità di sensori o comunicazioni.

Il limite è la scarsa reattività. Una volta fissato il ciclo, il verde viene assegnato anche in assenza di domanda e non può essere trasferito immediatamente a una coda inattesa. Il confronto sperimentale deve tuttavia essere formulato con cautela: battere un particolare piano statico non equivale a dimostrare la superiorità rispetto a ogni possibile piano ottimizzato. Nella campagna Manhattan il riferimento è il `program0` nativo della mappa, non un piano coordinato ottimizzato a posteriori sui casi di test.

2.1.2 Controllo attuato e adattivo

I controller attuati usano rivelatori per prolungare o terminare una fase sulla base della presenza veicolare. Regole di *gap-out*, ad esempio, chiudono il verde quando l'intervallo tra passaggi supera una soglia [CF12]. I sistemi adattivi operano a un livello più ampio: stimano lo stato corrente e aggiornano durate, cicli o selezione delle fasi. OPAC e SCOOT sono esempi storici di strategie sensibili alla domanda [Gar83; RB91].

Le soluzioni centralizzate possono coordinare una rete, ma richiedono più informazioni, comunicazione e capacità di calcolo. Le soluzioni decentralizzate riducono tale dipendenza e risultano naturalmente scalabili. Il Max-Pressure appartiene a quest'ultima famiglia: l'azione di ciascun semaforo dipende principalmente dalle code sulle corsie incidenti.

2.2 Strategie Max-Pressure

Il Max-Pressure deriva dalle politiche di backpressure impiegate per stabilizzare reti di code [TE92]. Applicato al traffico, confronta la quantità di veicoli che desidera attraversare l'incrocio con la capacità residua delle corsie riceventi. L'intuizione è servire i movimenti che riducono maggiormente il differenziale di coda senza trasferire indiscriminatamente congestione a valle.

2.2.1 Principio di funzionamento

Sia $\mathcal{M}$ l'insieme dei movimenti di un semaforo. Un movimento $m = (i, j)$ collega una corsia in ingresso i a una corsia in uscita j . Nella forma adottata nel progetto, la pressione del movimento è

$$p_{ij}(t) = q_i(t) - q_j(t), \quad (2.1)$$

in cui q_i e q_j sono le code osservate all'istante t . Se $\mathcal{M}_\phi$ è l'insieme dei movimenti abilitati dalla fase principale ϕ , la pressione della fase è

$$P_\phi(t) = \sum_{(i,j) \in \mathcal{M}_\phi} p_{ij}(t). \quad (2.2)$$

Il candidato naturale è quindi

$$\phi^*(t) = \arg \max_{\phi \in \Phi} P_\phi(t). \quad (2.3)$$

La sottrazione della coda downstream distingue il Max-Pressure da una politica che osserva soltanto gli ingressi: un movimento perde priorità se conduce a una corsia già occupata. Formulazioni più generali includono turning ratio, pesi, portate di saturazione o stime dei tassi di svolta; alcuni lavori hanno studiato anche versioni che non richiedono tassi di instradamento noti a priori [Gre+14]. La scelta semplificata adottata è coerente con l'obiettivo di ottenere un controller locale interpretabile e applicabile alle mappe disponibili.

2.2.2 Pressione dei movimenti e delle fasi

La pressione non coincide con la sola lunghezza della coda. Una fase può avere molti veicoli in ingresso ma una pressione modesta quando anche le uscite sono congestionate. Al contrario, una coda più corta può risultare prioritaria se può essere scaricata verso corsie libere. Il punteggio di fase somma i movimenti consentiti; ciò permette di valutare con un unico valore configurazioni semaforiche che servono più approcci contemporaneamente.

In una rete regolare il calcolo è completamente locale e può essere ripetuto a ogni passo. Le proprietà teoriche di throughput del Max-Pressure sono state studiate in reti di intersezioni e in implementazioni decentralizzate [Var13; Won+12; Le+15; Kou+14]. Studi di simulazione mostrano tuttavia che parametri, sensori e struttura della rete influenzano le prestazioni osservate [SY18; MUH20].

2.2.3 Proprietà e limiti del controllo locale

L'uso di informazioni locali rende l'algoritmo scalabile e robusto alla mancanza di un coordinatore centrale. Una variazione di domanda modifica immediatamente i differenziali di coda, senza richiedere la stima preventiva di un intero profilo giornaliero.

Esistono però quattro limiti principali. Primo, il criterio ideale non attribuisce automaticamente un costo al cambio di fase. Secondo, le code misurate non descrivono sempre la capacità fisica residua di una corsia. Terzo, massimizzare il servizio complessivo può ritardare ripetutamente un movimento debole. Quarto, decisioni localmente corrette possono non produrre coordinamento lungo un corridoio. La revisione di Levin evidenzia proprio la distanza tra proprietà teoriche e dettagli necessari per implementazioni pratiche [Lev23]; per questo la letteratura recente studia anche versioni cicliche, cycle-based, coordinate o basate sullo stato regionale della rete [And+18; LHO20; XBL24; LG24].

2.3 Estensioni del Max-Pressure

Le varianti studiate nel progetto non sono controller indipendenti dal punto di vista concettuale. Ognuna modifica il punteggio o il momento della decisione per affrontare un limite specifico. Questo rende possibile una valutazione per ablation: si mantiene il nucleo Max-Pressure e si osserva il contributo della singola estensione.

2.3.1 Costo delle transizioni semaforiche

Durante giallo e all-red nessuna delle direzioni principali utilizza pienamente l'intersezione. Cambi troppo frequenti possono quindi ridurre la capacità nonostante ogni scelta sia

localmente giustificata. Le strategie *lost-time-aware* introducono una soglia: la nuova fase deve offrire un guadagno sufficiente a compensare il servizio perso durante la transizione. La considerazione esplicita dello switching loss è un tema rilevante anche in formulazioni apprese del Max-Pressure [Wan+22]; le versioni cicliche e cycle-based affrontano un problema affine vincolando gli aggiornamenti a una struttura più vicina ai piani semaforici tradizionali [And+18; LHO20].

Un minimo verde svolge una funzione simile, ma in modo temporale: impedisce di rivalutare immediatamente la fase appena attivata. Un massimo verde costituisce invece una salvaguardia contro permanenze eccessive e forza periodicamente una nuova decisione.

2.3.2 Prevenzione dello spillback

Lo spillback si verifica quando la coda occupa la corsia fino a bloccare l'intersezione a monte. In questa condizione il solo conteggio dei veicoli fermi può reagire troppo tardi o non distinguere correttamente la capacità residua. Le strategie capacity-aware usano occupazione, lunghezza della corsia o stime di capacità per evitare di inviare veicoli verso un'uscita quasi piena [Gre+15; Noa+21]. Altri approcci pesano anche la posizione dei veicoli lungo l'arco, così da rappresentare meglio la propagazione spaziale dello spillback [LJ19].

Nel progetto vengono valutate due forme. La penalità downstream riduce in modo continuo il punteggio dei movimenti diretti verso corsie occupate. Il blocco anti-spillback è invece una protezione discreta: oltre una soglia di attivazione una fase viene esclusa, e torna disponibile soltanto sotto una soglia di rilascio. L'isteresi evita oscillazioni quando l'occupazione si trova vicino al limite.

2.3.3 Fairness e prevenzione della starvation

Una politica orientata al throughput può continuare a favorire flussi forti. Per limitare la starvation, al punteggio di una fase può essere aggiunto un bonus crescente con il tempo trascorso dall'ultimo servizio. Il bonus non sostituisce la pressione, ma rende progressivamente più competitivo un movimento trascurato. Il compromesso tra throughput, ritardo e fairness è discusso anche in varianti delay-based del Max-Pressure [Wu+18; LG22].

2.3.4 Gestione dei plotoni di veicoli

I veicoli non raggiungono necessariamente l'incrocio in modo uniforme. Il rilascio di un semaforo a monte può generare un plotone, cioè una sequenza di passaggi con headway ridotto. Interrompere il verde a metà del plotone spreca parte dell'opportunità di servizio e può ricreare rapidamente una coda. La platoon extension osserva gli intervalli di passaggio vicino alla stop line e, entro un limite massimo, prolunga la fase quando il flusso rimane

compatto. Questa logica è affine ai criteri di gap-out dei controller attuati, ma viene integrata nella decisione Max-Pressure.

2.4 Simulazione microscopica del traffico

La valutazione di un controller richiede di rappresentare non soltanto flussi aggregati, ma anche partenze, accelerazioni, cambi di corsia e interazioni tra veicoli. Per questo il progetto impiega una simulazione microscopica.

2.4.1 SUMO

SUMO¹, *Simulation of Urban MObility*, è un simulatore open source, microscopico e a passi discreti, progettato per reti stradali di dimensione urbana [Lop+18; Ecl26a]. Le reti sono descritte tramite file XML contenenti archi, corsie, connessioni, incroci e programmi semaforici. La simulazione produce informazioni per veicolo e consente di eseguire esperimenti senza interfaccia grafica, requisito essenziale per campagne batch.

Nel progetto SUMO gestisce la dinamica veicolare e la validità delle transizioni semaforiche. Il codice Python non sostituisce il simulatore: ne modifica le azioni attraverso un'interfaccia di controllo e raccoglie le osservazioni necessarie.

2.4.2 Interfaccia TraCI

TraCI², *Traffic Control Interface*, espone un protocollo con cui un processo esterno può interrogare e comandare una simulazione SUMO [Weg+08; Ecl26b]. A ogni passo il controller legge, tra le altre grandezze, veicoli fermi, occupazione delle corsie, stato delle fasi e passaggi su rivelatori virtuali. Può quindi richiedere il passaggio a una fase o a un programma semaforico differente.

Questa separazione è utile scientificamente: il modello del traffico rimane quello di SUMO, mentre la politica di controllo è implementata in Python e può essere modificata senza rigenerare la rete.

2.5 Situazione iniziale del progetto

Per chiarezza metodologica è opportuno distinguere quanto era già disponibile da quanto è stato realizzato nel lavoro. La ricostruzione è stata effettuata attraverso la cronologia Git e il confronto tra il commit iniziale dell'applicazione e la versione finale.

¹<https://www.eclipse.org/sumo/>

²<https://sumo.dlr.de/docs/TraCI/index.html>

2.5.1 Componenti software già disponibili

La base iniziale comprendeva:

- un runner capace di avviare SUMO e avanzare la simulazione;
- la classe astratta comune ai controller;
- un controller a tempi fissi;
- un generatore di popolazioni YAML;
- strutture per veicoli, tipi e metriche elementari;
- utility per risolvere i percorsi delle mappe;
- una raccolta di reti sintetiche e reali, tra cui Manhattan e Bologna.

Era quindi già presente l'infrastruttura minima per inserire veicoli e interagire con i semafori. Non erano invece disponibili un Max-Pressure completo, le varianti specialistiche, una campagna automatizzata di ablation e la verifica esplicita dei viaggi non conclusi.

2.5.2 Limiti della soluzione iniziale

L'esecuzione di una singola simulazione non è sufficiente per un confronto sperimentale. Mancavano una matrice dichiarativa dei casi, isolamento degli output, esecuzione parallela, controllo preventivo delle configurazioni, aggregazione su seed e report delle differenze rispetto a una baseline. La sola media sui veicoli arrivati, inoltre, poteva nascondere un'elevata censura.

Dal lato del controllo, un template intelligente non risolveva il problema della mappatura tra segnali SUMO, corsie e movimenti, né la gestione sicura di giallo e all-red. Questi aspetti sono risultati più complessi del calcolo della sola pressione e hanno richiesto una macchina a stati esplicita.

2.5.3 Contributi sviluppati nella tesi

L'evoluzione del repository del progetto³ mostra una progressione incrementale: prima il Max-Pressure puro, quindi spillback, lost-time-aware, fairness, Nmin, penalità downstream e plotoni; successivamente la pipeline di ablation, la correzione delle metriche e i controller ibridi per Bologna. La scelta di aggiungere una funzione alla volta ha permesso di attribuire i risultati e di individuare estensioni inattive o controproducenti. I capitoli seguenti descrivono questa architettura e l'implementazione risultante.

³Il codice sorgente, le configurazioni e i risultati sperimentali sono disponibili all'indirizzo: <https://github.com/Lugozzi04/sumo-traffic-control>.

3 Requisiti e progettazione del sistema

Un esperimento sui controller semaforici è credibile soltanto se la piattaforma evita confronti non omogenei e conserva abbastanza informazioni per riprodurre ogni risultato. Da questa esigenza derivano i requisiti e l'architettura descritti nel capitolo. La progettazione separa la politica di controllo dalla generazione del traffico e dalla valutazione, in modo che una modifica di un componente non alteri implicitamente gli altri.

3.1 Requisiti funzionali

Il sistema deve permettere di:

1. selezionare una mappa SUMO e una popolazione di veicoli;
2. eseguire sia il programma semaforico nativo sia un controller Max-Pressure configurabile;
3. associare a ogni variante un insieme esplicito di parametri;
4. generare più livelli e distribuzioni di domanda usando seed noti;
5. avanzare la simulazione fino all'esaurimento dei viaggi o a un limite dichiarato;
6. salvare metriche per veicolo, metriche aggregate e contatori interni del controller;
7. costruire automaticamente il prodotto cartesiano tra mappe, domande, seed e scenari;
8. eseguire più casi in parallelo senza collisioni tra file;
9. produrre classifiche e differenze rispetto a una baseline scelta.

Un requisito ulteriore, emerso durante lo sviluppo, è la capacità di rilevare errori prima di avviare campagne lunghe. La fase di preflight deve quindi controllare nomi delle mappe, scenari, file di rete, opzioni dei controller e percorsi di output.

3.2 Requisiti non funzionali

Oltre ai valori numerici, sono rilevanti *riproducibilità*, *modularità* e *osservabilità*. Questi requisiti hanno guidato scelte quali la serializzazione delle popolazioni, l'uso di configurazioni risolte e l'introduzione di contatori per le regole specialistiche.

3.2.1 Riproducibilità

Due controller devono essere confrontati sugli stessi veicoli, con le stesse route, gli stessi istanti di partenza e le stesse caratteristiche di guida. Per ogni terna mappa–domanda–seed

viene quindi generato un unico file di popolazione, riutilizzato da tutti gli scenari. Il seed compare sia nel nome sia nel contenuto del file. La directory di una campagna conserva inoltre la configurazione effettivamente risolta, evitando che un valore predefinito modificato in seguito renda ambiguo un risultato precedente.

La riproducibilità non implica che seed diversi producano gli stessi valori; serve piuttosto a rendere controllata la variabilità. Nella valutazione finale su Manhattan ogni scenario viene eseguito sui medesimi cinque seed e quindi sullo stesso insieme di popolazioni.

3.2.2 Modularità

Il simulatore, il controller, il generatore della domanda e la reportistica hanno responsabilità distinte. Tutti i controller rispettano un'interfaccia comune di collegamento e aggiornamento; il runner non deve conoscere la formula specifica con cui viene scelta una fase. Analogamente, la pipeline batch costruisce comandi per una simulazione singola senza duplicarne la logica.

Le estensioni del Max-Pressure sono abilitate da flag e parametri. Questo consente di mantenere un solo nucleo implementativo e di definire scenari sperimentali come configurazioni, anziché creare copie divergenti del controller.

3.2.3 Osservabilità

Una differenza nelle metriche finali non rivela da sola il comportamento che l'ha prodotta. Il sistema registra quindi numero di switch, attivazioni del massimo verde, passi di hold Nmin, eventi di blocco e rilascio spillback, prolungamenti per plotone e bonus di fairness. Se una variante coincide numericamente con il controller base e il suo contatore rimane a zero, è possibile concludere che il meccanismo non è stato sollecitato, non che abbia fallito dopo essersi attivato.

Ogni worker scrive inoltre un log separato. Il file di progresso aggrega casi completati, riusciti e falliti, rendendo possibile controllare una campagna eseguita in background.

3.3 Architettura generale

La figura 3.1 mostra i componenti principali della piattaforma sperimentale. Lo schema non elenca tutti i file sorgente, ma rappresenta i passaggi logici di una campagna: preparazione dei casi, costruzione della popolazione, esecuzione delle simulazioni, controllo semaforico, raccolta degli output e aggregazione finale.

Nel codice, la pipeline sperimentale è realizzata da `run_ablation.py`. Questo script risolve la combinazione di mappe, domande, seed e scenari, genera o recupera la popolazione

da usare e avvia un runner per ogni caso. `generate_population.py` non è mostrato come blocco separato nella figura, perché il suo risultato è il blocco *Popolazione YAML*: l'input di traffico che viene poi passato al runner.

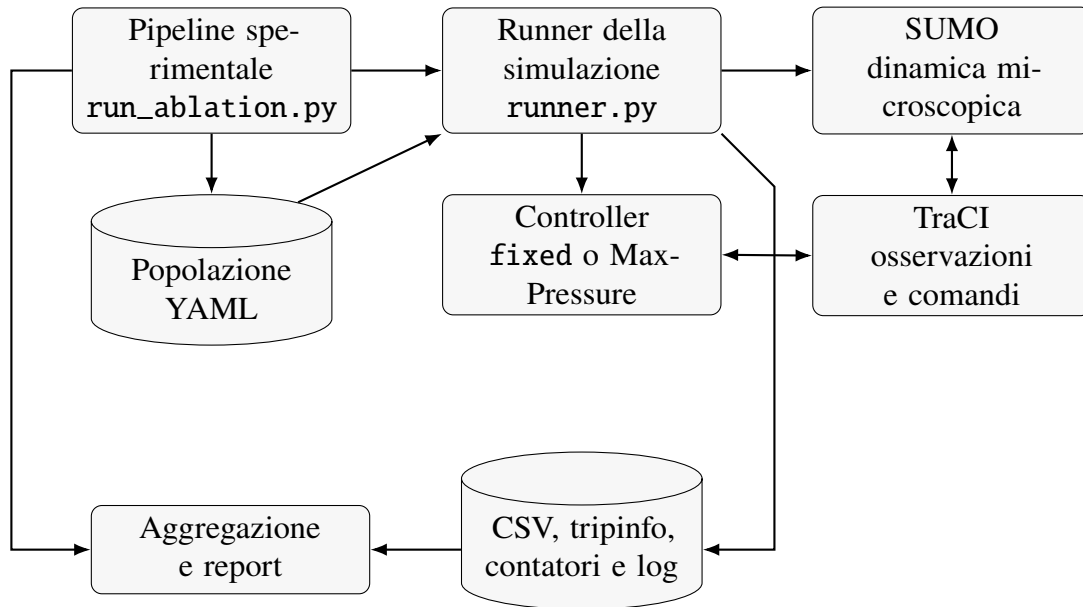


Figura 3.1. Architettura della piattaforma sperimentale.

La figura va quindi letta da sinistra verso destra. La pipeline prepara gli input e coordina l'esecuzione; il runner avvia SUMO, carica la popolazione e collega il controller mediante TraCI. Gli output del singolo caso vengono poi raccolti e, al termine della campagna, la pipeline li combina nei report finali.

3.3.1 Runner delle simulazioni

`runner.py` è il punto di ingresso per un singolo caso. Il modulo interpreta gli argomenti, individua i file della mappa, avvia SUMO, carica la popolazione e istanzia il controller richiesto. Il ciclo principale continua finché `getMinExpectedNumber()` segnala veicoli presenti o ancora attesi, oppure finché viene raggiunto il limite opzionale di passi.

A ogni iterazione il runner avanza SUMO e invoca l'aggiornamento del controller. SUMO rimane responsabile della dinamica microscopica, quindi aggiorna veicoli, corsie, code e completamento dei viaggi; TraCI è invece il canale attraverso cui il codice Python osserva lo stato della simulazione e invia comandi ai semafori. Alla chiusura il runner legge il file `tripinfo`, combina le informazioni di viaggio con i contatori online e scrive i risultati del caso. Questa sequenza centralizza il ciclo di vita della simulazione e garantisce che `fixed` e `Max-Pressure` differiscano soltanto nella politica semaforica.

3.3.2 Generazione delle popolazioni

La popolazione YAML è l'input di traffico di una simulazione. Contiene i veicoli da inserire nella rete, le route assegnate, gli istanti di partenza e i parametri di guida. `generate_population.py` costruisce questo file selezionando route valide dalla mappa e generando veicoli secondo il livello di domanda e il seed scelti. Durante l'esecuzione, `src/population.py` traduce poi la popolazione YAML in comandi TraCI.

La separazione tra generazione e simulazione impedisce che due scenari consumino in ordine diverso lo stesso generatore casuale. Il traffico non viene rigenerato per ogni controller: una data popolazione è un input immutabile del confronto.

3.3.3 Controller semaforici

La classe base definisce il contratto comune. Il controller `fixed` può lasciare in esecuzione un programma esistente o modificarne selettivamente il verde. Il controller `Max-Pressure` costruisce, per ogni semaforo, una rappresentazione di fasi principali, movimenti, corsie e stato temporale. Tutti i semafori della mappa vengono collegati all'avvio e aggiornati in modo decentralizzato.

La configurazione del `Max-Pressure` è parametrica. Gli scenari non contengono codice, ma combinazioni di flag quali `-lost-time-aware`, `-nmin-dynamic` o `-spillback-block`. Questa scelta ha reso possibile correggere il nucleo una sola volta e confrontare varianti che condividono la medesima gestione delle transizioni.

3.3.4 Raccolta e aggregazione delle metriche

Le metriche online descrivono lo stato e le decisioni del controller; quelle post-simulazione provengono dai viaggi completati. Il blocco *CSV, tripinfo, contatori e log* della figura raggruppa output con ruoli diversi: `tripinfo` è prodotto da SUMO e viene usato per derivare le metriche sui viaggi; i CSV e i riepiloghi conservano i valori elaborati dalla piattaforma; i contatori descrivono eventi interni del controller; i log aiutano a diagnosticare anomalie o fallimenti. Terminati i casi, la pipeline combina poi i risultati in:

- `run_results.csv`, una riga per simulazione;
- `summary_by_group.csv`, statistiche aggregate per scenario;
- `summary_vs_base.csv`, differenze rispetto alla baseline;
- `summary.md`, tabella leggibile della campagna;
- `winners.txt` e `table.txt`, classifiche sintetiche;
- `config_resolved.yaml`, configurazione effettiva.

Sebbene il formato generico possa contenere campi prodotti da SUMO per analisi di altro tipo, la tesi aggrega e interpreta esclusivamente metriche di mobilità e completezza dei viaggi.

3.4 Flusso di esecuzione di una simulazione

Il flusso completo è il seguente:

1. la pipeline risolve mappe, scenari, livelli di domanda e seed;
2. per ogni combinazione mappa–distribuzione–domanda–seed genera o recupera dalla cache una popolazione;
3. un worker prepara una directory isolata e avvia il runner;
4. il runner avvia un processo SUMO con i file della mappa;
5. i veicoli vengono inseriti secondo il file YAML;
6. il controller analizza i programmi semaforici e costruisce il proprio stato interno;
7. a ogni passo SUMO aggiorna veicoli e corsie, quindi il controller osserva lo stato e può richiedere una nuova fase;
8. al termine vengono chiusi TraCI e SUMO e viene analizzato il `tripinfo`;
9. il worker registra successo o fallimento e aggiorna il progresso;
10. completati tutti i casi, i risultati vengono raggruppati per mappa, distribuzione, domanda e scenario.

La generazione delle popolazioni precede l'esecuzione parallela. In questo modo nessun worker modifica un input condiviso mentre altri casi sono in corso.

3.5 Scelte progettuali

3.5.1 Separazione tra controller e simulatore

Si è scelto di non incorporare la logica adattiva nei file XML dei programmi semaforici. Il controller Python osserva e comanda SUMO mediante TraCI. Il vantaggio è la facilità di sperimentazione e di strumentazione, lo svantaggio è un costo di comunicazione a ogni passo. Per le dimensioni studiate, la maggiore chiarezza e modificabilità prevalgono su tale costo.

La separazione consente inoltre di preservare le transizioni definite nella mappa. Il controller sceglie una fase principale target, ma non sintetizza arbitrariamente una stringa di segnali potenzialmente insicura.

3.5.2 Configurazione mediante parametri

Le varianti sono definite in uno *scenario pack*. Ogni scenario associa un nome stabile al controller e ai flag. La configurazione dichiarativa riduce gli errori nei comandi lunghi, permette di riportare esattamente i parametri nei risultati e rende semplice escludere combinazioni non pertinenti. Per la campagna Manhattan sono state scelte una variante rappresentativa per ciascuna famiglia, anziché tutte le combinazioni esplorate durante il tuning.

3.5.3 Gestione delle mappe reali e sintetiche

Le mappe sintetiche hanno topologia regolare e sono adatte a isolare il comportamento algoritmico. Le mappe reali contengono numeri diversi di corsie, fasi eterogenee, svolte, segmenti brevi e piani semaforici specifici. Il layer di risoluzione dei percorsi e l'analisi dinamica dei programmi evitano di codificare nel controller gli identificativi di una singola rete.

Non tutte le strategie sono tuttavia trasferibili senza adattamento. Un programma statico specifico della mappa, come `program0` di Bologna, non ha un equivalente semantico universale. Per questo i capitoli sperimentali separano la valutazione generale su Manhattan dagli sviluppi specifici della rete reale.

4 Implementazione dei controller Max-Pressure

Il calcolo della pressione occupa poche righe, la parte più delicata consiste nel trasformare quel calcolo in azioni compatibili con programmi semaforici eterogenei. Il capitolo descrive prima la rappresentazione dei semafori e la macchina a stati, quindi le estensioni valutate. Le formule corrispondono alla versione effettivamente utilizzata nella campagna Manhattan.

4.1 Rappresentazione dei semafori

All'avvio il controller interroga TraCI per ottenere tutti i semafori, le logiche disponibili e i link controllati. Per ciascun semaforo costruisce una struttura contenente numero e durata delle fasi, fasi principali, movimenti per fase, tempi di attesa interni e stato della transizione. La costruzione non presuppone identificativi specifici della mappa.

In SUMO ogni semaforo della mappa può avere più programmi semaforici. Un programma non è quindi globale per l'intera rete, ma descrive il piano di uno specifico incrocio: fasi, durate e transizioni. Ogni programma viene identificato da un valore come 0, 1 o 2. Questi identificativi sono locali alla mappa e al semaforo: il programma 0 può indicare il piano nativo di un incrocio, mentre in un'altra rete o in un altro semaforo gli stessi numeri possono riferirsi a logiche diverse.

Per questo il controller non sceglie staticamente, ad esempio, sempre il programma 1. Nel Max-Pressure generale seleziona il programma con il maggior numero di fasi principali utilizzabili, preferendo quello già attivo in caso di parità. Le durate vengono lette e conservate, ma non sono il criterio con cui si sceglie il programma.

4.1.1 Identificazione delle fasi principali

Una fase è considerata principale se la stringa di stato contiene almeno un segnale verde, g o G, e non contiene segnali gialli. Le fasi solo gialle o rosse sono trattate come transizioni. Questa distinzione è necessaria perché il Max-Pressure deve scegliere tra configurazioni di servizio, non tra gli stati intermedi richiesti per passare in sicurezza da una configurazione all'altra.

La classificazione è ricavata dalla logica SUMO e non da una lista scritta a mano. Ciò permette allo stesso controller di operare su incroci con numeri di fasi e lunghezze delle stringhe di segnale diversi.

4.1.2 Costruzione dei movimenti

`getControlledLinks()` associa ogni posizione della stringa semaforica a uno o più collegamenti tra corsia in ingresso e corsia in uscita. Per ogni fase principale il controller visita le posizioni verdi e costruisce l'insieme

$$\mathcal{M}_\phi = \{(i, j) \mid \text{il movimento } i \rightarrow j \text{ è verde nella fase } \phi\}. \quad (4.1)$$

Eventuali duplicati vengono eliminati. Il risultato è una relazione esplicita tra una fase e le corsie sulle quali misurare code, occupazione e passaggi. Se una fase non abilita alcun movimento valido non partecipa alla selezione.

4.1.3 Conservazione delle transizioni di sicurezza

Il controller non passa direttamente da una stringa verde a un'altra. Quando seleziona un target, avanza alla fase successiva prevista dalla logica e memorizza il target come `pending_target`. Durante giallo e all-red attende la durata configurata nel programma SUMO. Se la sequenza raggiunge una fase principale diversa dal target, effettua il riallineamento soltanto al termine della transizione.

Questa soluzione conserva le durate di sicurezza della mappa e separa due decisioni: *quale* fase servire e *come* raggiungerla. Un meccanismo di recupero interviene se il semaforo rimane per almeno 20 secondi in uno stato non principale, condizione considerata anomala. Questa salvaguardia è stata introdotta dopo aver osservato stalli in campagne iniziali.

4.2 Controller Max-Pressure di base

4.2.1 Misura delle code

La coda su una corsia è misurata mediante `getLastStepHaltingNumber()`, cioè il numero di veicoli considerati fermi nell'ultimo passo. La misura viene memorizzata in una cache valida per il passo corrente, perché la stessa corsia può comparire in più movimenti. Si indica con $q_l(t)$ il valore associato alla corsia l .

La scelta del numero di veicoli fermi, anziché del numero totale presente, rende il punteggio sensibile alla congestione effettiva vicino all'intersezione. Non rappresenta tuttavia

in modo completo veicoli in avvicinamento o capacità fisica residua, le estensioni downstream e platoon sono state introdotte anche per compensare questa limitazione.

4.2.2 Calcolo della pressione

Per ogni movimento (i, j) si applica l'equazione (2.1). La pressione della fase è la somma dell'equazione (2.2). In assenza di estensioni, l'algoritmo produce inoltre la domanda della fase

$$D_\phi(t) = \sum_{(i,j) \in M_\phi} q_i(t), \quad (4.2)$$

utilizzata dal minimo verde dinamico e, nei controller successivi, per classificare il regime di traffico.

Una pressione negativa è ammessa: indica che le uscite sono più cariche degli ingressi. Se tutte le fasi valide hanno un punteggio finito, viene comunque scelta la migliore in senso relativo. Nel caso anti-spillback una fase può ricevere valore $-\infty$ e viene esclusa.

Questa implementazione conserva la struttura locale upstream–downstream delle formulazioni decentralizzate del Max-Pressure [Var13; Won+12].

4.2.3 Selezione della fase

Siano S_ϕ i punteggi, uguali alla pressione salvo bonus o penalità. La fase migliore cambia il semaforo soltanto se

$$\phi^* \neq \phi_c \quad \wedge \quad S_{\phi^*} > S_{\phi_c} + M, \quad (4.3)$$

in cui ϕ_c è la fase corrente e M il margine di switch. Nel controller base $M = 0$. La disuguaglianza stretta evita cambi in caso di pareggio. Se la condizione è soddisfatta, il controller registra lo switch, imposta il target pendente e avvia la sequenza di transizione.

4.2.4 Vincoli di minimo e massimo verde

Il valore predefinito del minimo verde è 10 secondi. Prima di tale durata la fase non viene rivalutata. Il massimo verde è 120 secondi e opera come limite di sicurezza: se esiste una fase alternativa finita, il controller può forzare il cambio quando il limite è raggiunto. Minimo e massimo non determinano quindi un ciclo fisso, ma delimitano la frequenza delle decisioni adattive.

È importante distinguere questo comportamento da un piano statico. Il controller *fixed* mantiene una durata prestabilita indipendentemente dalle code. Il Max-Pressure mantiene la fase almeno per il minimo, poi può conservarla o cambiarla in base ai punteggi osservati.

4.3 Macchina a stati del controller

Per ogni semaforo vengono mantenuti fase principale attiva, target pendente, tempo di permanenza in stati non principali, hold Nmin, tempo di svuotamento e prolungamento per plotone. Queste variabili rendono il controller una macchina a stati, non una funzione priva di memoria.

4.3.1 Fase attiva e target pendente

Nello stato ordinario il semaforo si trova in una fase principale. Il controller misura la rete e può restare oppure scegliere un target. Quando sceglie un target diverso, entra nello stato di transizione e non prende nuove decisioni di pressione fino al completamento della sequenza. In questo modo un punteggio che cambia durante il giallo non interrompe o inverte la manovra.

Raggiunto il target, lo stato attivo viene aggiornato e i temporizzatori della nuova fase vengono inizializzati. I tempi di attesa delle fasi non servite sono mantenuti soltanto quando è abilitata la fairness.

4.3.2 Gestione di giallo e all-red

La durata persa nel lasciare una fase è calcolata sommando le durate delle fasi non principali che la seguono fino alla prossima fase principale:

$$L_{\phi_c} = \sum_{k \in \mathcal{T}(\phi_c)} d_k, \quad (4.4)$$

con $\mathcal{T}(\phi_c)$ insieme degli stati di transizione e d_k pari alla relativa durata. Il valore viene riutilizzato sia dal margine lost-time-aware sia dal calcolo Nmin. Non viene quindi assunto un giallo uguale per tutti gli incroci.

4.3.3 Recupero dalle transizioni anomale

Due controlli evitano possibili stalli. Se esiste un target pendente e una fase di transizione termina, il controller avanza o si riallinea al target. Se invece non esiste un target ma il semaforo rimane in una fase non principale oltre la soglia di 20 secondi, viene ripristinata

la prima fase principale valida. Il recupero non è una politica di traffico: è una protezione operativa contro stati incoerenti o logiche di rete non previste.

4.4 Lost-Time-Aware

La variante lost-time-aware richiede che il guadagno di punteggio superi una stima del costo della transizione. Il margine complessivo è

$$M = \varepsilon_{abs} + \varepsilon_{rel}|S_{\phi_c}| + \gamma s L_{\phi_c}, \quad (4.5)$$

in cui γ è il guadagno lost-time, s una portata di saturazione espressa in veicoli al secondo e L_{ϕ_c} il tempo perso. La variante finale Manhattan usa $\gamma = 0,60$ e $s = 0,50$. Il margine rende esplicito il costo del cambio di fase, problema affrontato anche nelle formulazioni Max-Pressure che considerano lo switching loss [Wan+22].

L'effetto atteso è ridurre switch marginali. Un valore eccessivo può tuttavia rendere il controller lento a reagire e prolungare fasi non più ottimali. Per questo l'efficacia non può essere dedotta dalla sola plausibilità della regola: deve essere misurata insieme al numero di switch e agli interventi del massimo verde.

4.5 Minimo verde dinamico N_{min}

N_{min} stima quanti veicoli sia opportuno servire prima di abbandonare una fase. Dato il costo di transizione dell'equazione (4.4), calcola

$$N_{cost} = \alpha s L_{\phi_c}, \quad (4.6)$$

$$N_{target} = \max\{N_{floor}, N_{cost}, gD_{\phi_c}\}, \quad (4.7)$$

in cui α pesa il costo, N_{floor} è un minimo di veicoli, g il coefficiente dipendente dalla domanda e D_{ϕ_c} la domanda della fase. Se la domanda è positiva, N_{target} viene limitato a non superarla. La durata di hold è

$$H_{\phi_c} = \max\left\{G_{min}^N, \frac{N_{target}}{s}\right\}. \quad (4.8)$$

Se la fase si svuota prima del termine, un timer consente il rilascio anticipato invece di attendere inutilmente. La configurazione studiata usa $\alpha = 0,80$, $N_{floor} = 2$, $G_{min}^N = 4$ secondi, $g = 0,25$ e rilascio dopo un secondo di domanda nulla.

Questa configurazione modifica due aspetti: abilita l'hold dinamico e riduce da 10 a 4 secondi il minimo verde di base. Nell'interpretare i risultati occorre quindi evitare di attribuire automaticamente ogni guadagno alla formula Nmin. I contatori mostrano, infatti, che nel traffico basso l'hold dinamico non viene quasi mai attivato; il beneficio deriva soprattutto dalla maggiore rapidità di rivalutazione.

4.6 Fairness

Per ogni fase non servita che presenta domanda viene incrementato un tempo di attesa interno w_ϕ . Quando la fase è attiva, o quando la sua domanda si annulla, il valore viene azzerato. Il bonus è

$$B_\phi(w_\phi) = \mu \frac{w_\phi}{w_\phi + w_{1/2}}, \quad (4.9)$$

ed è aggiunto alla pressione. μ è il bonus asintotico e $w_{1/2}$ il tempo al quale viene raggiunta metà del bonus massimo. La configurazione Manhattan usa $\mu = 5$ e $w_{1/2} = 20$ secondi.

La funzione cresce rapidamente all'inizio e poi satura. Non garantisce un ordine rigido delle fasi, ma rende progressivamente più difficile ignorare un movimento con domanda persistente. Il contatore registra quante volte è stato applicato un bonus positivo e la somma complessiva dei bonus. La regola affronta così lo stesso compromesso fra throughput e fairness studiato nei controller basati sul ritardo [Wu+18; LG22].

4.7 Penalità downstream

La pressione di base sottrae già la coda a valle. La variante downstream aggiunge una penalità continua basata sull'occupazione filtrata con media mobile esponenziale:

$$\bar{o}_j(t) = \eta o_j(t) + (1 - \eta) \bar{o}_j(t - 1), \quad (4.10)$$

$$p'_{ij}(t) = q_i(t) - q_j(t) - \beta \bar{o}_j(t). \quad (4.11)$$

La campagna usa $\beta = 0,4$ e $\eta = 0,30$. Il filtro riduce la sensibilità a picchi di un solo passo. La penalità è morbida: una corsia occupata rende il movimento meno attraente, ma non lo esclude. Il rischio è penalizzare anche una normale occupazione in reti lontane dalla saturazione, introducendo un costo senza prevenire alcun blocco reale.

4.8 Anti-spillback

Il blocco rigido usa due stime della saturazione downstream. La scelta di considerare la capacità finita delle corsie segue il principio degli approcci capacity-aware e anti-spillback [Gre+15; Noa+21]. La prima stima è l'occupazione filtrata $\bar{o}_j$; la seconda approssima il riempimento della capacità di stoccaggio:

$$f_j = \min \left\{ 1, \frac{q_j}{\max(1, \ell_j/7,5)} \right\}, \quad z_j = \max\{\bar{o}_j, f_j\}, \quad (4.12)$$

in cui ℓ_j è la lunghezza della corsia in metri e 7,5 metri è lo spazio medio in coda assunto per veicolo. Una corsia non bloccata entra nello stato di blocco quando $z_j \geq \theta_{on}$ e sono presenti almeno h_{min} veicoli fermi; rimane bloccata finché $z_j \geq \theta_{off}$. Poiché $\theta_{off} < \theta_{on}$, la regola possiede isteresi.

Nella configurazione `on90_off80`, $\theta_{on} = 0,90$, $\theta_{off} = 0,80$ e $h_{min} = 3$. Se anche un solo movimento di una fase conduce a una corsia bloccata, l'intera fase riceve pressione $-\infty$. È una scelta conservativa: protegge la rete dal trasferimento di congestione, ma può sottrarre servizio anche agli altri movimenti della stessa fase. I contatori di blocco, rilascio e passi bloccati servono a verificare se la protezione sia mai entrata in funzione.

4.9 Platoon extension

Per ogni corsia in ingresso alla fase attiva viene collocato un rivelatore virtuale a 8 metri dalla fine. Il controller registra gli istanti in cui i veicoli attraversano tale posizione e calcola la media degli ultimi quattro headway. Un plotone è riconosciuto se la media non supera la soglia configurata e l'ultimo passaggio non è più vecchio del tempo di gap-out.

Quando il controller sta per rivalutare la fase, la presenza del plotone può prolungare il verde fino a un massimo aggiuntivo. La configurazione finale usa soglia e gap-out di 2,1 secondi e un massimo di 4 secondi. Un controllo downstream impedisce l'estensione solo quando sono contemporaneamente abilitate le misure di spillback o penalità downstream. Nello scenario standalone `mp_platoon_x4` tali moduli non sono attivi; di conseguenza il parametro di guardia dichiarato non modifica il comportamento. Questo dettaglio implementativo è rilevante per non sovrainterpretare il test.

4.10 Margini e stabilità dello switch

Oltre al termine lost-time, il controller supporta un margine assoluto e uno relativo. Con la sola componente relativa,

$$M = \varepsilon_{rel} |S_{\phi_c}|. \quad (4.13)$$

Lo scenario Manhattan `mp_switch_rel005` usa $\varepsilon_{rel} = 0,05$. L'obiettivo è introdurre una banda morta: una fase alternativa deve migliorare il punteggio corrente di almeno il 5%. Anche qui il vantaggio potenziale, meno oscillazioni, si contrappone al rischio di ritardare una decisione utile.

4.11 Contatori di attivazione

Il controller espone un dizionario di statistiche, successivamente aggregato nel CSV di ciascuna esecuzione. Per i test dei primi sei capitoli vengono utilizzati:

- numero di switch decisi dal margine e numero di switch forzati dal massimo verde;
- passi in cui `Nmin` ha mantenuto la fase;
- eventi di blocco, rilascio e passi con `spillback` attivo;
- passi di estensione dovuti a un plotone;
- numero e somma dei bonus di fairness positivi.

Queste misure non sono obiettivi da massimizzare. Un numero elevato di attivazioni può indicare che la regola affronta un fenomeno frequente, ma anche che interviene troppo aggressivamente. La loro funzione è spiegare le metriche di mobilità e verificare la coerenza tra configurazione e comportamento.

5 Metodologia e piattaforma sperimentale

La qualità di una politica semaforica non può essere stabilita da una sola simulazione. Il protocollo deve controllare domanda, seed, durata e completezza dei viaggi, oltre a rendere confrontabili gli output. Il capitolo definisce la pipeline sperimentale, i modelli di domanda, le metriche e le minacce alla validità. La configurazione specifica di Manhattan è presentata nel capitolo 6.

5.1 Obiettivi della valutazione

La valutazione risponde a quattro domande:

1. il Max-Pressure riduce attesa e tempo di viaggio rispetto al programma statico nativo?
2. il vantaggio rimane osservabile al crescere della domanda?
3. quali estensioni migliorano il controller base e in quale regime?
4. il comportamento interno conferma che la regola specialistica si è effettivamente attivata?

Per rispondere, si adotta un'analisi di ablazione (*ablation*). Con questo termine si intende un confronto in cui si parte da un modello di riferimento e si aggiunge, oppure si isola, una sola modifica alla volta. In questo lavoro il riferimento interno è `mp_base`: ogni scenario Max-Pressure abilita una sola famiglia di estensioni rispetto a esso. In questo modo, se una variante migliora o peggiora, il cambiamento può essere ricondotto con maggiore chiarezza alla regola aggiunta.

Le popolazioni sono identiche tra controller e cambiano soltanto tra seed o livello di domanda. Il piano fisso basato su `program0` serve a confrontare il controllo adattivo con un semaforo non adattivo già presente nella mappa, mentre `mp_base` serve a capire quanto valore aggiungano le singole estensioni Max-Pressure.

5.2 Pipeline sperimentale

`utils/run_ablation.py` automatizza la campagna. A partire da insiemi di mappe, domande, distribuzioni, seed e scenari, costruisce tutti i casi, prepara gli input, esegue i processi e aggrega gli output. Questa automazione è stata necessaria perché comandi manuali lunghi

avevano prodotto, nelle prime prove, errori difficili da diagnosticare e risultati non sempre omogenei.

5.2.1 Generazione dei casi sperimentali

Il numero totale di casi è

$$N_{case} = N_{mappe} N_{set} N_{domande} N_{seed} N_{scenari}. \quad (5.1)$$

Il termine N_{set} indica eventuali varianti nella generazione della popolazione; quando non sono previste, vale 1.

Prima dell'avvio, la pipeline risolve i nomi delle mappe e degli scenari, calcola i preset di domanda effettivi e scrive la configurazione. Per ciascuna combinazione di mappa, set, domanda e seed genera una sola popolazione. Tutti i controller riferiti a quella combinazione ricevono lo stesso file.

Gli scenari non vengono definiti manualmente a ogni lancio, ma sono raccolti in gruppi predefiniti, chiamati *scenario pack*. Ogni scenario del pack associa un nome leggibile ai flag effettivi passati al runner; in questo modo, leggendo i risultati, è possibile risalire alla configurazione usata senza ricostruirla a posteriori.

5.2.2 Organizzazione delle campagne

Ogni campagna riceve un identificativo del tipo `run_NNNN_AAAAMMGG_hhmmss`. La directory contiene popolazioni, risultati aggregati e una sottodirectory per ciascun caso. Il file di progresso riporta il totale pianificato e il conteggio di successi e fallimenti.

Questa organizzazione preserva la provenienza dei risultati e consente di confrontare campagne successive senza mescolarne i file. I report finali usati nel capitolo 6 provengono interamente da una sola campagna completa.

5.2.3 Report automatici

Per ogni gruppo mappa-set-domanda-scenario il report aggrega le metriche sui seed disponibili. Se x_s è il valore di una metrica nel seed s , vengono calcolate la media e la deviazione standard campionaria:

$$\bar{x} = \frac{1}{n} \sum_{s=1}^n x_s, \quad (5.2)$$

$$s_x = \sqrt{\frac{1}{n-1} \sum_{s=1}^n (x_s - \bar{x})^2}. \quad (5.3)$$

La deviazione standard è detta campionaria perché i seed rappresentano solo un campione delle possibili popolazioni generate. Il report calcola poi, per ogni scenario, la differenza percentuale rispetto alla baseline b :

$$\Delta_x = 100 \frac{\bar{x} - \bar{x}_b}{\bar{x}_b}. \quad (5.4)$$

Per attesa e tempo di viaggio un valore negativo indica un miglioramento. Le classifiche usano come criterio primario il tempo medio di attesa, ma la conclusione viene controllata sulle altre metriche e sulla censura.

5.3 Generazione della domanda di traffico

Una popolazione contiene identificativo, route, istante di partenza e tipo per ogni veicolo. La finestra di generazione è distinta dalla durata totale della simulazione: dopo l'ultimo veicolo inserito, la simulazione continua finché i veicoli in rete terminano il viaggio, salvo un limite esplicito.

I tipi sono estratti con probabilità 75% autovetture, 10% veicoli per consegne, 10% motocicli, 3% camion e 2% autobus. L'eterogeneità influenza lunghezza, accelerazione, distanza minima e dinamica di partenza, rendendo la popolazione meno artificiale di un insieme composto da sole autovetture.

5.3.1 Livelli *low*, *medium* e *high*

I livelli *low*, *medium* e *high* indicano intensità relative, non valori universali. La pipeline usa valori calibrati per le mappe note; altrimenti stima *medium* dal numero di route e ricava *low* e *high* come 60% e 140%. Per reti con oltre 650 route si ottengono 2400, 4000 e 5600 veicoli, con partenze in $[0, 3600]$ secondi.

Questa calibrazione rende i livelli proporzionati alla varietà dei percorsi della mappa. Non garantisce però che *high* produca saturazione o spillback: la gravità effettiva dipende anche da capacità, lunghezza delle route e concentrazione spaziale. Nel capitolo dei risultati tale distinzione risulta essenziale.

5.3.2 Seed e riproducibilità

Un unico oggetto pseudo-casuale, inizializzato dal seed, determina partenze, route e tipi. La popolazione viene poi ordinata per istante di partenza e serializzata. Nella campagna Manhattan si usano i seed da 1 a 5. Poiché ogni file è condiviso da tutti gli scenari, le differenze tra controller non dipendono da un diverso campione di veicoli.

5.4 Profilo di guida `human_light`

Il profilo `human_light` riduce l'idealizzazione del comportamento senza introdurre una taratura aggressiva. La velocità iniziale dei veicoli non è sempre quella massima, ma viene scelta in modo più realistico da SUMO. Per ciascun tipo vengono specificati tempo di reazione τ , imperfezione di guida σ , intervallo di azione, gap alle precedenze e ritardo di partenza. La tabella 5.1 riporta i parametri principali usati per ciascun tipo di veicolo.

Tabella 5.1. Parametri principali del profilo `human_light`.

Tipo	τ [s]	σ	Azione [s]	Gap [s]	Startup [s]
Autovettura	1,45	0,55	1,0	1,40	0,80
Consegna	1,55	0,50	1,0	1,45	1,00
Camion	1,75	0,45	1,0	1,60	1,20
Autobus	1,75	0,45	1,0	1,60	1,20
Motociclo	1,25	0,65	1,0	1,25	0,45

Il profilo è identico per tutti i controller. Non cerca di riprodurre una specifica popolazione cittadina, ma evita partenze istantanee e reazioni perfette che potrebbero favorire irrealisticamente switch molto frequenti.

5.5 Controller di confronto

5.5.1 `fixed_program0`

Lo scenario `fixed_program0` forza `program0`, cioè il programma con `programID=0`. SUMO esegue poi ciclicamente le durate definite nella mappa. Il controller non osserva code e non modifica il piano: per questo è la baseline principale di Manhattan, cioè il funzionamento semaforico nativo senza controllo adattivo.

5.5.2 **fixed_tuned**

La piattaforma supporta anche `fixed_tuned`, che forza a 30 secondi le fasi principali di `program0` mantenendo le transizioni. Questa baseline è stata utile nelle campagne esplorative su Bologna, ma non viene inclusa nella campagna Manhattan finale: il suo parametro non deriva da un'ottimizzazione specifica della griglia e avrebbe aggiunto un confronto non necessario al primo studio di ablazione.

5.6 **Metriche**

5.6.1 **Tempo di attesa**

Per ciascun veicolo SUMO fornisce il tempo totale trascorso in attesa. La media è l'indicatore primario perché misura direttamente la conseguenza delle scelte semaforiche. Viene riportato anche il 95-esimo percentile, utile a verificare che una buona media non nasconda una coda di utenti fortemente penalizzati.

5.6.2 **Tempo di viaggio e time loss**

Il tempo di viaggio è la differenza tra arrivo e partenza. La *time loss* è la perdita rispetto a una percorrenza ideale secondo il modello SUMO e comprende rallentamenti, arresti e interazioni. Le due metriche non sono intercambiabili: route più lunghe aumentano il viaggio anche a parità di controllo, mentre il time loss isola meglio l'inefficienza incontrata.

5.6.3 **Velocità media**

Per ogni viaggio completato la velocità media è il rapporto tra distanza e durata; il report aggrega poi i veicoli. Un controller efficace dovrebbe produrre, a parità di route, velocità maggiori coerentemente con minori attese e perdite di tempo.

5.6.4 **Viaggi completati e tasso di completamento**

Il numero di viaggi completati misura quanti veicoli raggiungono la destinazione. Rapportato alla popolazione pianificata fornisce il tasso di completamento; rapportato alla durata osservata può invece contribuire a una misura di throughput. Quando la simulazione prosegue fino allo svuotamento, tutti gli scenari dovrebbero completare la stessa popolazione e il solo conteggio finale non distingue le prestazioni. In presenza di un limite temporale, il numero di arrivi diventa invece essenziale: un controller che lascia veicoli bloccati non può essere valutato soltanto sui pochi arrivati.

5.6.5 Censura

Siano N_p i viaggi pianificati e N_c quelli completati. La censura è

$$C = \frac{N_p - N_c}{N_p}. \quad (5.5)$$

Una censura positiva indica che le metriche di viaggio sono calcolate su un sottoinsieme. Non equivale automaticamente a un errore, ma rende il confronto più delicato e deve essere riportata. La campagna Manhattan usa `max_steps=0`, cioè nessun cutoff imposto dalla pipeline, e termina con $C = 0$ in tutti i 135 casi.

5.7 Criteri di confronto

Il criterio primario è la minore attesa media, purché censura e numero di viaggi completati siano comparabili. Il confronto con il piano statico usa `fixed_program0`; il confronto tra estensioni Max-Pressure usa invece `mp_base`. Si controllano poi tempo di viaggio, time loss, 95-esimo percentile dell'attesa e velocità. Quando il vantaggio è piccolo o cambia tra seed, il risultato viene interpretato come vittoria aggregata, non come superiorità sostanziale.

Le attivazioni interne vengono usate come indizio causale debole ma utile. Ad esempio, risultati e contatori identici tra `spillback` e `mp_base` indicano che la soglia non è stata raggiunta. Non si conclude che lo `spillback` sia inutile in generale; si conclude che la campagna non ha testato il regime per cui è stato progettato.

6 Valutazione su Manhattan 8×8

La rete Manhattan costituisce il primo banco di prova completo perché riduce le peculiarità topologiche e consente di osservare il comportamento dei controller in una griglia ripetibile. Il capitolo presenta configurazione, risultati e contatori della campagna finale. L'obiettivo non è scegliere un unico parametro universale, ma capire quali meccanismi producano un beneficio prima di passare alla rete reale.

6.1 Motivazione dello scenario sintetico

Una rete reale combina contemporaneamente segmenti corti, svolte, incroci con fasi diverse e domanda spazialmente non uniforme. Se una variante peggiora, può essere difficile stabilire quale caratteristica abbia causato il risultato. Manhattan 8×8 offre invece incroci quasi omogenei, elevata semaforizzazione e molte route. È quindi adatta a un'analisi di ablazione controllata.

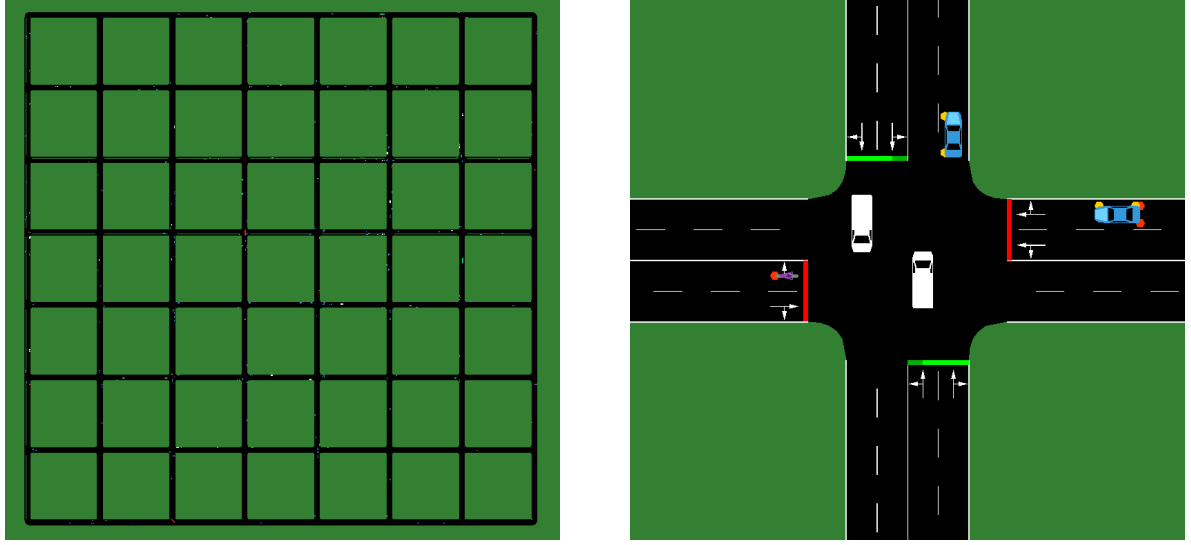
La regolarità non rende il test banale. Sessanta semafori prendono decisioni locali e le code si propagano attraverso la griglia. Il piano fisso, inoltre, non usa offset coordinati: tutti i programmi partono con offset zero. La campagna misura dunque quanto l'adattamento locale riesca a sfruttare una domanda diffusa rispetto alla ripetizione dello stesso ciclo.

6.2 Topologia della rete

La mappa `manhattan8x8_100pc` contiene 64 giunzioni non interne, delle quali 60 semaforizzate e quattro angoli regolati a priorità. I 224 archi non interni hanno complessivamente 448 corsie, due per arco. Le lunghezze delle corsie sono comprese tra 139,2 e 143,2 metri, con media circa 139,5 metri. Il file delle route contiene 800 percorsi, da 1 a 15 archi e con lunghezza media pari a 7,276 archi.

La figura 6.1 mostra la rete in SUMO e un dettaglio di intersezione. La struttura regolare permette di isolare l'effetto della logica semaforica: ogni incrocio è simile agli altri e le differenze di prestazione dipendono soprattutto dal controller, non da particolarità geometriche locali.

Ogni semaforo possiede due logiche. `program0`, usato da `fixed_program0`, segue un ciclo statico: $42 + 3 + 3 + 42 + 3 + 3 = 96$ secondi. La seconda logica mantiene le stesse transizioni, ma lascia il cambio fase al controller Max-Pressure. I 30 secondi di verde principale al posto dei 42 riguardano solo `fixed_tuned`.



(a) Vista complessiva della griglia.

(b) Dettaglio di un incrocio semaforizzato.

Figura 6.1. Scenario Manhattan 8×8 visualizzato in SUMO.

6.3 Configurazione sperimentale

La campagna finale è identificata da `run_0028_20260612_131724`¹. Comprende una mappa, una distribuzione uniforme, tre domande, cinque seed e nove scenari:

$$1 \times 1 \times 3 \times 5 \times 9 = 135 \text{ simulazioni.} \quad (6.1)$$

Le partenze sono distribuite uniformemente nei primi 3600 secondi. La domanda *low* prevede 2400 veicoli, *medium* 4000 e *high* 5600. Non è imposto un numero massimo di passi: ciascun caso prosegue fino a esaurimento. Tutti i casi sono terminati con successo; in ciascuno il numero di viaggi completati coincide con quello pianificato e la censura è zero. Non sono stati rilevati nei log avvisi di collisione, teletrasporto o frenata d'emergenza.

La tabella 6.1 riassume gli scenari. I valori non sono tutte le combinazioni provate durante lo sviluppo, ma una configurazione definitiva per ogni famiglia.

6.4 Confronto tra Max-Pressure e controllo fisso

La campagna Manhattan conferma il risultato più netto della tesi: tutte le varianti Max-Pressure riducono l'attesa rispetto al programma fisso. Il `mp_base` riduce l'attesa del

¹I risultati completi della run sono disponibili nel repository in `logs/ablation/runs/run_0028_20260612_131724`: in particolare `table.txt`, `summary_by_group.csv`, `summary_vs_base.csv` e `run_results.csv`.

Tabella 6.1. Scenari della campagna Manhattan.

Scenario	Configurazione caratterizzante
fixed_program0	Programma statico program0 della mappa
mp_base	Minimo 10 s, massimo 120 s, margine nullo
mp_lta_g060_sf050	$\gamma = 0,60$, $s = 0,50$ veicoli/s
mp_nmin_a080_f2	$\alpha = 0,80$, floor 2, minimo 4 s, $g = 0,25$
mp_downstream_b04	$\beta = 0,4$, EMA $\eta = 0,30$
mp_spillback_on90_off80	Soglie 0,90/0,80, almeno 3 veicoli fermi
mp_fair_mu5_w20	$\mu = 5$, $w_{1/2} = 20$ s
mp_platoon_x4	Headway e gap 2,1 s, estensione massima 4 s
mp_switch_rel005	Margine relativo 5%

59,46%, 55,93% e 52,66% nei tre livelli; il miglior specialista raggiunge 64,26%, 60,09% e 53,05%. La figura 6.2 riassume il confronto senza riportare nel testo tutte le combinazioni numeriche generate dalla pipeline.

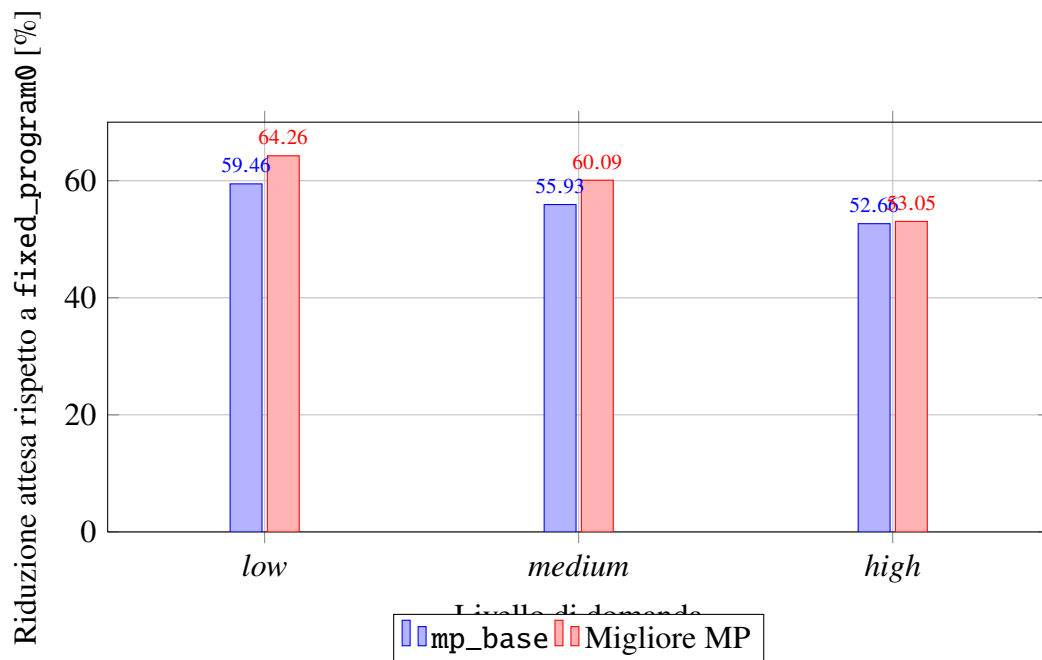


Figura 6.2. Riduzione percentuale del tempo medio di attesa su Manhattan.

Lo scenario migliore è mp_nmin_a080_f2 in *low* e *medium*, mentre in *high* è mp_fair_mu5_w20. Il valore assoluto dell'attesa passa da 65,85 a 23,54 secondi in *low*, da 66,89 a 26,70 in *medium* e da 67,75 a 31,81 in *high*. La censura è nulla in tutti i casi, quindi il confronto non dipende da viaggi incompleti.

Anche i percentili alti confermano il miglioramento: il 95-esimo percentile scende da 137,22 a 48,00 secondi in *low*, da 138,80 a 54,20 in *medium* e da 141,00 a 66,00 in *high*. Il risparmio medio rispetto al piano statico è ampio: 42,31 secondi in *low*, 40,20 in *medium* e 35,94 in *high*.

6.5 Valutazione delle singole estensioni

Il confronto con `fixed_program0` dimostra il valore del controllo adattivo nella griglia, ma non implica che ogni estensione migliori `mp_base`. In Manhattan molte varianti restano molto vicine tra loro: questo non le rende inutili, ma indica che la domanda uniforme non evidenzia tutti i fenomeni per cui sono state progettate.

6.5.1 Effetto chiaro: `Nmin`

`Nmin` è l'estensione con il comportamento più distinguibile. Produce il miglior risultato in *low* e *medium*, con un risparmio su `mp_base` di 3,16 e 2,78 secondi. In *high* il vantaggio scende a 0,04 secondi, quindi è trascurabile rispetto alla variabilità tra seed.

I passi di hold `Nmin` sono in media 0 in *low*, 2,4 in *medium* e 23,2 in *high*. Il guadagno a basso traffico non nasce quindi dall'hold dinamico, ma soprattutto dal minimo verde ridotto a 4 secondi. In *high* l'hold entra in funzione, ma il beneficio sull'attesa è minimo e il tempo di viaggio peggiora leggermente.

6.5.2 Varianti quasi equivalenti a `mp_base`

Anti-spillback e margine relativo producono gli stessi valori di `mp_base` fino alla precisione dei report. Nel primo caso nessuna corsia raggiunge contemporaneamente il punteggio 0,90 e almeno tre veicoli fermi; nel secondo caso il 5% di banda morta non modifica le decisioni già prese dal controller.

Anche `platoon` e `fairness` restano molto vicini a `mp_base`. `Platoon` rileva arrivi compatti e aumenta i passi estesi al crescere della domanda, ma il vantaggio massimo sull'attesa è inferiore a 0,2 secondi. `Fairness` diventa nominalmente la migliore in *high*, ma con soli 0,26 secondi di margine su `mp_base`. Queste differenze sono troppo piccole per parlare di una superiorità stabile.

6.5.3 Varianti penalizzate dalla domanda uniforme

LTA e penalità downstream peggiorano rispetto a `mp_base`. LTA introduce un margine prudenziale contro gli switch frequenti, ma nella griglia uniforme le pressioni sono abbastanza

regolari e il costo della prudenza supera il beneficio. La penalità downstream, invece, non trova un vero regime di blocco a valle: reagisce a occupazioni normali e altera la scelta delle fasi senza prevenire uno spillback reale.

Questa lettura delimita il significato della campagna Manhattan. Anche 5600 veicoli con domanda uniforme sono sufficienti per confrontare Max-Pressure e piano fisso, ma non per valutare in modo completo estensioni pensate per code localizzate, corridoi dominanti, picchi temporali o saturazione a valle. Per questo la campagna permette di individuare i primi limiti dello scenario sperimentale Manhattan: il traffico è troppo omogeneo per mettere in crisi alcune estensioni. Nei capitoli successivi questi problemi vengono affrontati su Bologna, dove la topologia reale e le distribuzioni skewed e peak rendono il traffico più concentrato e variabile.

6.6 Analisi dei contatori di attivazione

La tabella 6.2 riassume i contatori più utili per interpretare le varianti. Le unità sono passi o eventi medi per simulazione; per `mp_base` viene riportata la coppia switch da margine/interventi da massimo verde.

Tabella 6.2. Contatori medi di attivazione per simulazione.

Meccanismo	<i>low</i>	<i>medium</i>	<i>high</i>
<code>mp_base</code> : switch / max-green	6071 / 91	8591 / 40	10184 / 21
LTA: switch / max-green	5273 / 260	7566 / 158	9106 / 90
Nmin: passi di hold	0,0	2,4	23,2
Fairness: bonus positivi	5584	7951	9538
Spillback: passi bloccati	0,0	0,0	0,0
Platoon: passi estesi	113,6	227,0	288,8

I contatori evitano due letture superficiali. Una regola può attivarsi spesso senza migliorare, come fairness e platoon; oppure può non attivarsi affatto, come spillback e margine relativo. In entrambi i casi il risultato numerico diventa interpretabile, invece di restare una semplice classifica.

6.7 Confronto complessivo

La campagna supporta con chiarezza l'ipotesi principale: su Manhattan 8×8 con domanda uniforme, il controllo Max-Pressure riduce in modo ampio attesa, viaggio e time loss rispetto a `program0` nativo, mantenendo censura nulla. Il `mp_base` da solo riduce l'attesa del

59,46%, 55,93% e 52,66% nei tre livelli. Non è quindi necessario un modulo specialistico per ottenere il beneficio fondamentale.

Tra le estensioni, Nmin è la scelta più efficace a domanda bassa e media, principalmente grazie al minimo verde più corto. A domanda alta fairness, platoon, Nmin e mp_base sono compresi entro 0,26 secondi di attesa. La classifica nominale non deve oscurare tale prossimità. LTA e downstream mostrano invece che aggiungere prudenza o informazione non produce automaticamente una politica migliore: una regola utile in congestione può costare capacità quando il fenomeno bersaglio è assente.

Il piano `fixed_program0` rimane quasi costante tra 65,85 e 67,75 secondi di attesa. Ciò non significa che il traffico non aumenti: viaggio e time loss peggiorano e la velocità scende. Significa piuttosto che il ciclo statico impone una componente dominante di ritardo anche in *low*, mentre il Max-Pressure evita verdi assegnati a movimenti con poca domanda. La rete regolare e l'assenza di offset coordinati favoriscono questo vantaggio locale. Non si può estendere la conclusione a un piano fisso ottimizzato per ogni profilo di domanda.

6.8 Indicazioni ricavate per lo scenario reale

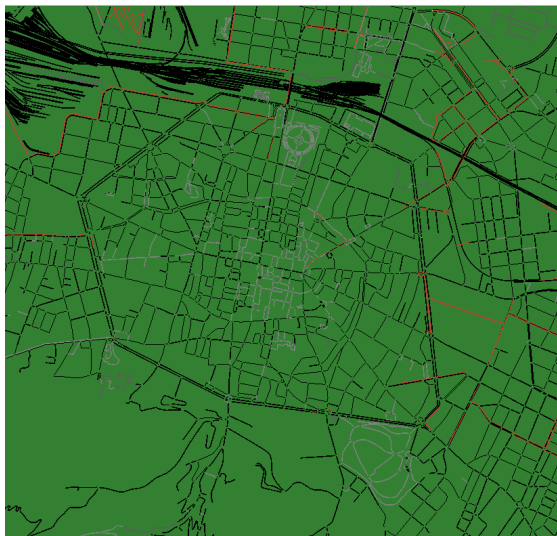
La campagna Manhattan chiarisce il ruolo del Max-Pressure di base: è già un riferimento forte e ogni estensione deve giustificare il proprio costo. Nmin mostra che ridurre il minimo verde può essere molto efficace quando il traffico è leggero, ma anche che il guadagno non coincide sempre con l'hold dinamico della formula. Spillback, al contrario, non può essere valutato davvero se la domanda non produce code a valle sufficientemente critiche.

Da questi risultati nasce il passaggio a Bologna. La griglia regolare dimostra che il controllo adattivo può funzionare, ma non riproduce corridoi dominanti, picchi temporali e programmi semaforici già calibrati sulla topologia. Nella rete reale il problema diventa quindi più selettivo: capire quando convenga usare Max-Pressure, quando conservare il piano nativo e quando attivare una variante specialistica.

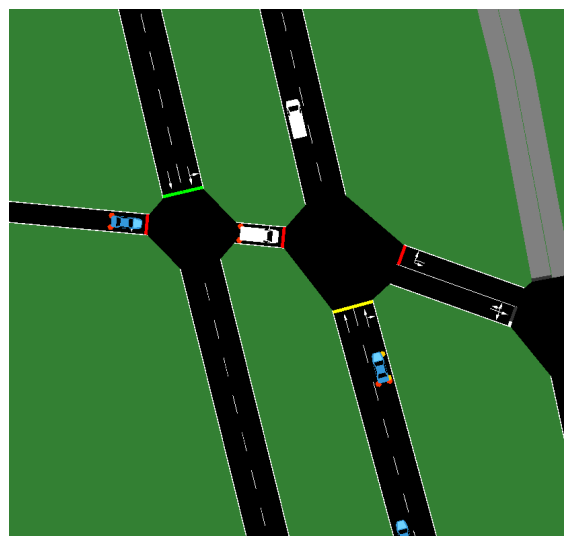
7 Adattamento alla rete di Bologna

7.1 Caratteristiche della rete

Il passaggio da Manhattan alla rete di Bologna cambia la natura del problema. La rete sintetica del capitolo precedente è regolare, simmetrica e completamente controllata; Bologna è invece una rete reale importata in SUMO, con incroci eterogenei, archi di lunghezza molto diversa, manovre non uniformi e piani semaforici già presenti nella mappa. Il file `bologna.net.xml` contiene 8 439 giunzioni totali, di cui 4 525 non interne, 10 281 archi stradali non interni, 10 896 corsie non interne e 228 giunzioni semaforizzate. Per ogni semaforo sono presenti due logiche, per un totale di 456 programmi e 2 216 fasi. La figura 7.1 mostra una vista complessiva della rete e un dettaglio di incrocio.



(a) Vista ampia della rete.



(b) Dettaglio di un incrocio reale.

Figura 7.1. Rete di Bologna visualizzata in SUMO.

Questa differenza strutturale è importante perché Max-Pressure assume una decisione locale: valuta la pressione delle fasi di un singolo incrocio e sceglie quella più conveniente nel breve periodo. In una griglia regolare questo principio è spesso sufficiente, perché gli incroci sono simili e i flussi sono distribuiti in modo abbastanza omogeneo. In una rete reale, invece, alcuni nodi hanno un ruolo molto più critico di altri, inoltre il programma statico può contenere durate e sequenze specifiche della topologia che un controller puramente locale non sfrutta.

Il dato più rilevante emerso durante il lavoro è che il programma semaforico nativo di Bologna, indicato nei risultati come `fixed_program0`, non è una baseline debole. Al contrario, si è dimostrato competitivo soprattutto con domanda leggera e media. Per questo la parte finale della tesi non si limita a verificare se Max-Pressure batta un piano fisso qualunque, ma studia come combinare adattività locale e riuso del piano nativo.

7.2 Problemi riscontrati

7.2.1 Complessità della topologia

Il primo problema riguarda la topologia. In Bologna non tutti gli incroci possono essere trattati come copie di uno stesso schema. Il numero di fasi, le direzioni servite, la presenza di corsie corte e la distanza fra incroci variano molto. Di conseguenza una stessa regola Max-Pressure può avere effetti diversi: in alcuni punti riduce rapidamente una coda, in altri può spostare il problema verso una corsia downstream già vicina alla saturazione.

Per questo motivo sono stati introdotti contatori di attivazione e metriche di diagnosi. I report non registrano soltanto attesa e tempo di viaggio, ma anche viaggi completati, viaggi non completati, censura, eventi di spillback, passi di blocco downstream, passi di estensione platoon e contatori interni del meta-controller. Queste misure permettono di capire se un miglioramento è reale oppure se deriva da una selezione dei soli veicoli arrivati.

7.2.2 Stalli e congestione

Un secondo problema è la propagazione della congestione. Se un incrocio manda veicoli verso una corsia downstream già piena, l'effetto locale può essere negativo anche quando la pressione a monte è alta. Le prime versioni dei meccanismi downstream e spillback erano troppo rigide: in alcuni casi non si attivavano mai, in altri diventavano così invasive da ridurre la capacità utile.

La soluzione adottata è stata separare lo scenario spillback dal ramo `safe` di `mp_super`. Lo spillback standalone applica sempre un vincolo rigido: se una fase scarica veicoli verso una corsia a valle quasi bloccata, quella fase viene esclusa. Il ramo `safe`, invece, si attiva solo in emergenza e combina due protezioni: il blocco spillback delle fasi verso uscite sature e la penalità downstream, che riduce la pressione delle fasi verso uscite molto occupate ma non ancora bloccate. La soglia elevata adottata nella configurazione finale mantiene questo ramo come ultima risorsa, evitando che la prudenza downstream domini il comportamento ordinario del controller.

7.3 Preparazione della mappa e compatibilità

La rete di Bologna è stata utilizzata tramite i file presenti in `sumo_xml_files/bologna`. Durante la fase di preparazione sono stati controllati i programmi semaforici, le fasi principali e la compatibilità con TraCI. Una parte del lavoro software è consistita nel rendere il controller robusto rispetto a mappe con più logiche semaforiche. Gli scenari Max-Pressure puri ricavano le fasi controllabili dalla logica selezionata per il semaforo; `mp_program0` e `mp_super`, invece, riusano esplicitamente `program0` quando entrano in modalità statica.

7.4 Analisi del programma semaforico `program0`

La baseline `fixed_program0` forza `program0`, già presente nella mappa. Non compie decisioni online: ripete le durate e le fasi definite nel file SUMO. Su Bologna questo comportamento si è rivelato molto più competitivo di una baseline statica uniforme. Una spiegazione plausibile è che il programma nativo non assegni la stessa durata a tutti gli incroci, ma conservi configurazioni specifiche della rete. I risultati non permettono tuttavia di attribuire il vantaggio a una forma precisa di coordinamento.

La variante `fixed_tuned` è stata usata come controllo secondario: forza ancora `program0`, ma porta a 30 secondi la durata delle fasi verdi principali, cioè le fasi che contengono almeno un segnale verde e non sono transizioni gialle. La sequenza delle fasi e i tempi di giallo/all-red restano quelli del piano originale. I risultati mostrano che questa uniformazione peggiora rispetto al `program0` originale. Il piano nativo costituisce quindi un riferimento non banale e più appropriato di una temporizzazione identica per tutti gli incroci.

7.5 Limiti del Max-Pressure puro su Bologna

Il comportamento di `mp_base` su Bologna ha evidenziato tre limiti. Primo, il controller è locale e non conosce eventuali coordinamenti impliciti fra incroci. Secondo, il costo dello switch, già modellato dalle varianti con margine e lost-time-aware, diventa più visibile quando il traffico è leggero e le pressioni sono instabili. Terzo, la pressione misura code e downstream in modo reattivo, ma non distingue da sola fra traffico persistente, picchi temporanei e saturazione downstream.

In pratica, `mp_base` rimane una baseline fondamentale, ma non è sufficiente per Bologna. Nella campagna finale, su domanda uniforme *low* l'attesa media è 406,64 secondi contro 198,65 di `fixed_program0`; in domanda uniforme *medium* è 781,01 contro 562,67. Il divario non scompare aumentando la domanda: `mp_base` è peggiore del piano nativo in tutti i nove gruppi, con incrementi dell'attesa compresi tra il 27,37% e il 102,97%.

7.5.1 Prestazioni nel traffico leggero

Nel traffico leggero il vantaggio dell'adattività è ridotto. Le code sono piccole, molte fasi hanno pressioni simili e un cambio può costare più di quanto guadagni. In questo regime il `program0` nativo ha spesso una buona prestazione perché evita oscillazioni e assegna tempi stabili.

La scoperta è stata importante perché ha corretto un'ipotesi iniziale: non è vero che un controller dinamico debba essere sempre più veloce di uno fisso. Se il traffico non giustifica lo switch, la decisione migliore può essere non applicare Max-Pressure. Da questa osservazione nasce il controller ibrido `mp_program0`, descritto nella paragrafo 7.8.

7.5.2 Assenza di coordinamento esplicito

Max-Pressure è decentralizzato: ogni semaforo decide osservando il proprio intorno. Questo è un punto di forza dal punto di vista implementativo, perché non richiede un'ottimizzazione globale della rete, ma può diventare un limite in presenza di corridoi e successioni di incroci ravvicinati. Una decisione locale aggressiva può ridurre una coda e, nello stesso tempo, aumentare il carico dell'intersezione successiva.

Il progetto finale non introduce un coordinamento globale esplicito, ma affronta il problema in modo pragmatico: mantiene il piano nativo quando il carico è basso, usa Max-Pressure solo quando serve e attiva specialisti diversi in base ai segnali osservati.

7.6 Calibrazione della domanda

Per Bologna sono stati fissati i tre livelli di domanda riportati nella tabella 7.1.

Tabella 7.1. Livelli di domanda usati per Bologna.

Livello	Veicoli	Inizio [s]	Fine [s]
<i>low</i>	1500	0	3600
<i>medium</i>	3000	0	3600
<i>high</i>	5000	0	3600

La scelta nasce da test esplorativi: valori più bassi non stressavano la rete, mentre valori più alti producevano rapidamente congestione difficile da interpretare con simulazioni brevi. Il livello *high* non rappresenta una condizione ordinaria, ma un test di stress utile per capire se i meccanismi specialistici si attivano e se la censura resta sotto controllo.

7.7 Distribuzioni di domanda per Bologna

Per Bologna non basta variare il numero di veicoli. Le prime prove hanno mostrato che una domanda uniforme può non creare corridoi critici o picchi temporali sufficienti a distinguere i controller. Sono quindi usati tre set di popolazione.

7.7.1 **balanced**

Balanced campiona le route con la stessa probabilità e distribuisce le partenze uniformemente nel tempo. Non significa che ogni arco riceva lo stesso flusso: le route possono sovrapporsi, ma il generatore non introduce pesi aggiuntivi.

7.7.2 **skewed**

Skewed mantiene partenze uniformi, ma assegna a ogni route un peso

$$w_r = E_r^{1,4}, \quad (7.1)$$

dove E_r è il numero di archi della route. Le route più lunghe vengono quindi estratte più spesso e tendono a caricare maggiormente alcuni corridoi.

7.7.3 **peak**

Peak usa lo stesso campionamento spaziale di **skewed** e concentra anche le partenze. Ogni istante di partenza combina una componente beta, pesata all'80%, con una componente uniforme, pesata al 20%:

$$u_{peak} = 0,80 u_\beta + 0,20 u_{uniforme}. \quad (7.2)$$

La scelta della prima componente beta avviene con probabilità 0,55. In questo modo si simulano due ondate di domanda senza fissare manualmente le route più cariche: la scelta rimane casuale, ma riproducibile tramite seed.

7.8 Controller ibrido **mp_program0**

7.8.1 **Misura del carico locale**

mp_program0 è il primo controller ibrido introdotto per Bologna. La sua idea è semplice, quando il carico locale è basso, lascia lavorare **program0**, quando il carico supera

una soglia, passa a Max-Pressure. Il controller non coincide con le formulazioni cycle-based della letteratura, ma risponde alla stessa esigenza pratica: conservare una struttura semaforica stabile quando l'adattività istantanea non è necessaria [And+18; LHO20]. Il carico è calcolato dalla domanda delle fasi:

$$L(t) = \min \left(1, \frac{\sum_p d_p(t)}{M\lambda_{ref}} \right), \quad (7.3)$$

dove $d_p(t)$ è la domanda della fase p , M è il numero di movimenti ricavati dalle fasi principali dell'incrocio e λ_{ref} è il riferimento di carico. Nella configurazione usata per Bologna $\lambda_{ref} = 3,5$.

7.8.2 Passaggio tra `program0` e Max-Pressure

Il passaggio non avviene con una sola soglia. Per evitare oscillazioni sono usate due soglie:

- ingresso in Max-Pressure quando $L(t) \geq 0,60$;
- ritorno a `program0` quando $L(t) \leq 0,40$.

La modalità deve inoltre essere confermata per quattro aggiornamenti consecutivi. Questa scelta rende il controller più stabile: una variazione istantanea della coda non basta per cambiare politica. Il risultato è un controller che preserva il vantaggio del piano statico in *low* e prova a recuperare reattività quando il traffico cresce.

7.8.3 Isteresi e stabilità della selezione

L'isteresi è il punto centrale di `mp_program0`. Senza isteresi, il carico vicino alla soglia causerebbe continui passaggi fra fisso e adattivo, con un costo di switch elevato e un comportamento difficile da spiegare. Con due soglie, invece, una volta entrato in Max-Pressure il controller non torna subito a `program0` appena il carico scende leggermente.

La valutazione del capitolo successivo verifica questa idea sull'intera matrice sperimentale. Il risultato rilevante, anticipato qui soltanto in termini qualitativi, è che una regola ibrida semplice può rimanere competitiva anche quando il piano statico nativo è forte.

7.9 Meta-controller `mp_super`

7.9.1 Motivazione

`mp_super` nasce da una constatazione sperimentale: non esiste uno specialista Max-Pressure che vinca in tutti i regimi. In Manhattan le varianti sono vicine fra loro; in Bologna,

invece, i casi *high* mostrano differenze marcate e nessuna estensione standalone supera stabilmente il piano nativo. Il meta-controller cerca quindi di conservare `program0` quando è sufficiente e di scegliere online una famiglia Max-Pressure coerente con il regime locale.

Le modalità disponibili sono:

- `program0`, per condizioni leggere o poco informative;
- `base`, Max-Pressure standard;
- `lta`, variante lost-time-aware;
- `nmin`, variante con minimo verde dinamico forte;
- `fair`, variante con bonus di attesa;
- `platoon`, estensione del verde per arrivi compatti;
- `safe`, protezione downstream di emergenza.

7.9.2 Carico (load)

Il carico, indicato nel codice come *load*, è calcolato come in equazione (7.3), con riferimento 3,5. Nella versione finale le soglie sono:

- ingresso dai tempi fissi ai rami MP: $L \geq 0,38$;
- uscita verso `program0`: $L < 0,28$;
- regime *high*: $L \geq 0,62$;
- regime Nmin forte: $L \geq 0,72$.

La separazione fra soglia di ingresso e soglia di uscita evita ritorni immediati a `program0` quando il carico cala per pochi passi.

7.9.3 Squilibrio della domanda (imbalance)

L'imbalance misura quanto la domanda è concentrata su una fase rispetto alla media:

$$I(t) = \min \left(1, \frac{\max_p d_p(t) - \bar{d}(t)}{\bar{d}(t)} \right). \quad (7.4)$$

Se $I(t) \geq 0,50$, il controller considera il nodo sbilanciato. Questo segnale è importante per distinguere una congestione distribuita da una situazione in cui una fase specifica accumula molta più domanda delle altre.

7.9.4 Squilibrio dell'attesa (wait imbalance)

Il wait imbalance usa la stessa struttura di equazione (7.4), ma applicata ai tempi di attesa accumulati per fase. Serve a catturare una forma di ingiustizia temporale: anche

se la domanda istantanea non è estrema, una fase può essere stata penalizzata a lungo. La soglia finale è 0,50. Quando attesa e domanda sono entrambe sbilanciate, `mp_super` tende a selezionare LTA; quando l'attesa è sbilanciata ma la domanda non lo è, può selezionare fairness.

7.9.5 Variabilità temporale (burstiness)

La burstiness misura quanto cambia la domanda fra il passo corrente e quello precedente. Il codice combina due componenti: la variazione del totale e la variazione della distribuzione fra fasi. Il valore viene limitato in $[0, 1]$. La soglia finale è 0,10. Un valore elevato indica arrivi a ondate o cambiamenti rapidi, condizioni nelle quali Nmin può evitare di restare bloccato troppo a lungo su una fase non più dominante.

7.9.6 Rilevazione dei plotoni (platoon score)

Il platoon score controlla se sono stati osservati arrivi compatti sulla fase corrente. La versione finale usa una regola binaria coerente con lo scenario `mp_platoon_x2`: il punteggio vale 1 se l'headway rilevato è minore o uguale a 2,1 secondi, altrimenti vale 0. La soglia del super è 0,50, quindi la modalità platoon viene selezionata solo quando il rilevatore segnala davvero un arrivo compatto.

7.9.7 Saturazione downstream

Il downstream score è il massimo fra occupazione EMA e riempimento stimato delle corsie downstream servite dalla fase. Il ramo `safe` richiede downstream score almeno 0,97, carico almeno 0,72 e presenza di burstiness o wait imbalance. Quando si attiva, il controller evita fasi che manderebbero altri veicoli verso una corsia quasi satura e applica la penalità downstream. Lo scopo non è ridurre direttamente l'attesa media, ma impedire che Max-Pressure alimenti un blocco a valle.

7.9.8 Regole di selezione delle modalità

Le regole finali sono ordinate per priorità. `safe` viene valutato per primo, ma soltanto nelle condizioni estreme descritte sopra. Se il carico è sotto la soglia associata alla modalità corrente, il controller resta o torna a `program0`. Per carichi almeno pari a 0,72, burstiness seleziona Nmin, attesa e domanda entrambe sbilanciate selezionano LTA, la sola attesa sbilanciata seleziona fairness e un plotone compatto può selezionare platoon; in assenza di questi segnali viene comunque usato Nmin forte.

Fra 0,62 e 0,72 valgono regole specialistiche analoghe, ma la modalità di ripiego è base. In questo intervallo la burstiness può già selezionare `nmin`, mentre attesa e domanda sbilanciate selezionano `lta`, la sola attesa sbilanciata può selezionare `fair` e un plotone compatto può selezionare `platoon`. Sotto 0,62, invece, la burstiness da sola non attiva `nmin` e porta alla modalità base.

La logica può essere letta come un albero di decisione scritto a mano. Non è un modello appreso: le soglie derivano dalle campagne esplorative e dall'interpretazione dei contatori. Il vantaggio è la trasparenza; il limite è che la taratura resta manuale.

7.9.9 Isteresi temporale

Oltre all'isteresi sul carico, `mp_super` usa una conferma temporale: una nuova modalità deve essere proposta per quattro aggiornamenti consecutivi prima di essere adottata. Questo riduce il rumore decisionale. Nelle campagne esplorative i contatori di switch del super sono comunque elevati nei casi *high*, segno che la rete presenta molti regimi locali diversi. Nella campagna finale la conferma temporale, insieme alle soglie separate di ingresso e uscita, mantiene tuttavia `program0` come modalità dominante e limita gli interventi adattivi alle situazioni che superano stabilmente le soglie.

7.10 Evoluzione delle soglie di `mp_super`

L'evoluzione di `mp_super` è stata guidata dai risultati. Le prime versioni lasciavano inattivi alcuni rami. I contatori hanno quindi motivato l'aggiunta di `wait_imbalance`, `Nmin` forte e `LTA`, mentre la soglia downstream di `safe` è stata portata a 0,97 per limitarne l'uso alle condizioni prossime alla saturazione. Ogni modifica deriva così da una diagnosi osservabile, non dalla sola ricerca del risultato migliore.

8 Valutazione finale su Bologna

8.1 Protocollo sperimentale

La valutazione su Bologna verifica come i controller adattivi si comportino quando competono con un programma semaforico reale già presente nella mappa. Il confronto combina tre livelli di domanda, tre insiemi di popolazioni (*balanced*, *skewed* e *peak*) e tre seed. Gli insiemi variano la distribuzione spaziale dei viaggi e, nel caso *peak*, anche il profilo temporale delle partenze. Per ogni combinazione vengono analizzati dieci scenari: *fixed_program0*, *fixed_tuned*, *mp_base*, *mp_program0*, *mp_super* e le varianti *LTA*, *Nmin*, *spilloback*, *fairness* e *platoon*. La matrice considerata comprende quindi 270 simulazioni.

I risultati provengono dalla campagna *run_0029_20260614_143342*, eseguita con il pack *bologna_thesis_v1*, profilo di guida *human_light* e *max_steps=0*. L'assenza di un cutoff impone a ciascuna simulazione di proseguire fino al completamento dei viaggi. Tutti i casi analizzati sono terminati correttamente, con tre repliche disponibili e censura nulla.



Figura 8.1. Esempio di accumulo di veicoli in una simulazione su Bologna con distribuzione peak e domanda alta.

La figura 8.1 mostra una condizione diversa dalla griglia Manhattan: le code si concentrano su corridoi specifici, in presenza di archi corti e incroci ravvicinati. La durata non limitata delle simulazioni consente di osservare anche la fase di smaltimento della congestione e di calcolare le metriche sulla stessa popolazione completa per tutti i controller.

8.2 Risultati aggregati

La tabella 8.1 riporta, per ciascun gruppo, lo scenario con la minore attesa media fra quelli analizzati. Le deviazioni standard sono calcolate sui tre seed; le ultime due colonne misurano la variazione rispetto a `fixed_program0`. Un valore negativo indica un miglioramento.

Tabella 8.1. Miglior scenario per ciascun gruppo della campagna Bologna.

Set	Domanda	Scenario	Attesa [s]	Δ attesa	Δ viaggio
Balanced	<i>low</i>	<code>mp_program0</code>	$196,28 \pm 8,92$	-1,19%	-0,33%
Balanced	<i>medium</i>	<code>mp_super</code>	$333,88 \pm 68,97$	-40,66%	-28,32%
Balanced	<i>high</i>	<code>mp_super</code>	$913,58 \pm 44,75$	-23,58%	-18,41%
Skewed	<i>low</i>	<code>mp_super</code>	$248,32 \pm 12,15$	-41,47%	-24,72%
Skewed	<i>medium</i>	<code>mp_program0</code>	$714,80 \pm 52,30$	-38,59%	-28,08%
Skewed	<i>high</i>	<code>mp_program0</code>	$1702,46 \pm 94,73$	-35,55%	-26,82%
Peak	<i>low</i>	<code>mp_super</code>	$396,67 \pm 41,44$	-34,87%	-21,90%
Peak	<i>medium</i>	<code>mp_super</code>	$1097,53 \pm 50,43$	-31,72%	-22,60%
Peak	<i>high</i>	<code>mp_program0</code>	$2167,71 \pm 50,08$	-32,78%	-24,17%

I controller ibridi occupano il primo posto in tutti i gruppi: `mp_super` ottiene cinque vittorie e `mp_program0` quattro. Il vantaggio sul piano nativo è limitato in balanced *low*, dove la differenza fra `mp_program0` e `fixed_program0` è soltanto 2,37 secondi. Negli altri otto gruppi la riduzione dell'attesa del miglior ibrido è compresa fra il 23,58% e il 41,47%.

La figura 8.2 amplia il confronto includendo `mp_base`. Il grafico usa la variazione percentuale rispetto al piano nativo, così da rendere confrontabili gruppi con tempi assoluti molto diversi.

Il risultato separa nettamente due comportamenti. `mp_base` aumenta l'attesa in tutti i gruppi, dal 27,37% al 102,97%. Al contrario, `mp_program0` migliora il piano nativo in tutti i gruppi, mentre `mp_super` lo migliora in otto casi su nove e rimane vicino a `fixed_program0` in balanced *low*. Mediando le variazioni percentuali sui nove gruppi, `mp_program0` ottiene -28,84% e `mp_super` -30,27%.

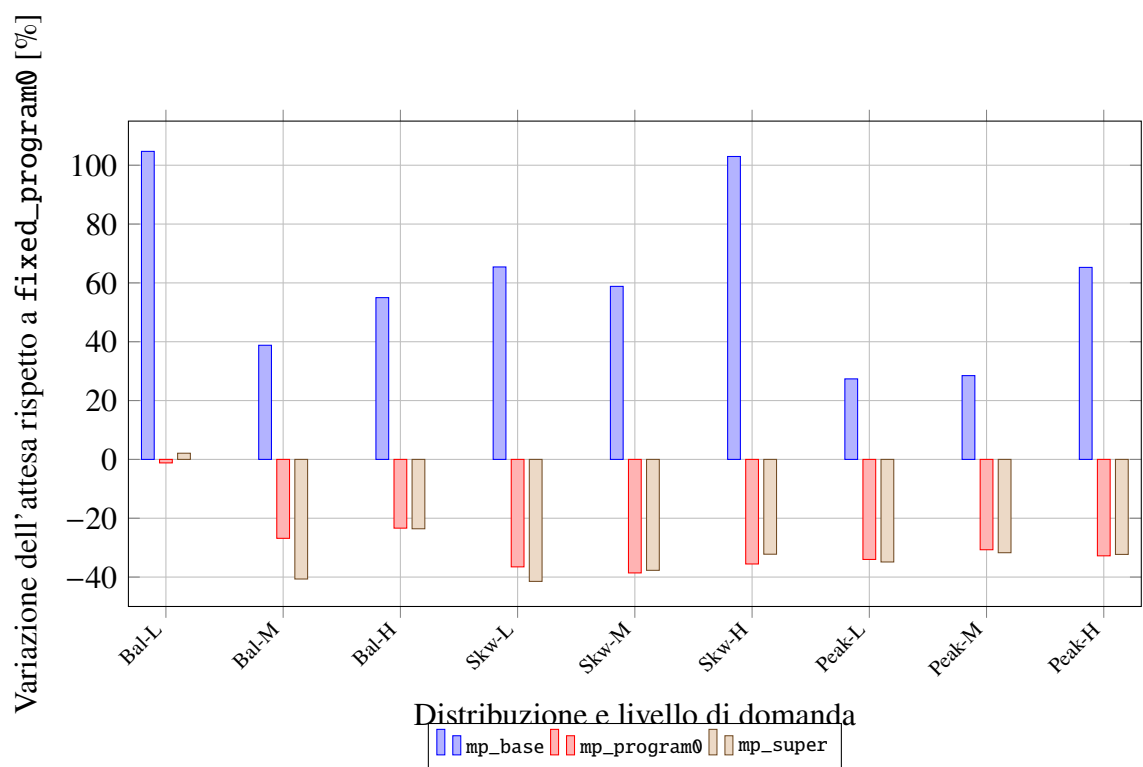


Figura 8.2. Variazione dell'attesa media rispetto a fixed_program0; valori negativi indicano un miglioramento.

8.3 Completezza e variabilità

La censura nulla risolve il principale limite delle campagne esplorative. Ogni scenario completa la popolazione pianificata, quindi attesa, tempo di viaggio e time loss sono calcolati sugli stessi veicoli. Il miglior controller per attesa riduce anche il tempo medio di viaggio in tutti i gruppi. Il percentile 95 dell'attesa migliora dal 3,46% in *balanced low* fino al 68,91% in *skewed low*: il vantaggio non deriva quindi dal sacrificio della coda lunga.

I dati aggregati mostrano che la classifica fra i due ibridi può variare quando le medie sono molto vicine. In *balanced high* le loro medie differiscono di soli 2,53 secondi; nei casi *peak medium* e *peak high* l'ordinamento cambia fra i seed. Questa variabilità non modifica la conclusione principale: nella matrice considerata il miglior risultato di gruppo è sempre ottenuto da `mp_program0` o da `mp_super`. I due controller risultano quindi complementari: il primo è più semplice, il secondo seleziona dinamicamente il ramo Max-Pressure più adatto ai segnali locali.

8.4 Comportamento delle varianti specialistiche

Le estensioni standalone non ottengono vittorie su Bologna. Fairness è la variante Max-Pressure pura con la minore attesa in tutti i nove gruppi, ma rimane sempre peggiore di `fixed_program0`. LTA e platoon si mantengono generalmente vicini a `mp_base`; il profilo Nmin forte risulta troppo aggressivo quando viene applicato come politica continua.

La campagna finale chiarisce anche il comportamento dello spillback. Il meccanismo si attiva in tutti i gruppi: gli eventi medi di blocco passano da 5 in *balanced low* a circa 2000 nei casi *skewed high* e *peak high*. L'attivazione non produce però un vantaggio, perché l'esclusione rigida di un'intera fase può sottrarre capacità anche quando soltanto uno dei suoi movimenti conduce a una corsia satura. Lo spillback non è quindi inattivo; è la sua applicazione continua a risultare troppo penalizzante. Anche `fixed_tuned` è sempre peggiore del programma nativo, confermando che uniformare a 30 secondi i verdi principali elimina informazioni utili presenti nella mappa.

8.5 Uso delle modalità di `mp_super`

Il comportamento del super è interpretabile perché, a ogni aggiornamento controllabile di un semaforo, il controller registra la modalità attiva. Si definisce qui *passo-modalità* la coppia formata da un incrocio controllato e da un aggiornamento del controller; il conteggio è quindi aggregato sia nel tempo sia sui semafori. La figura 8.3 mostra la quota di passi-modalità in cui il controller abbandona `program0`.

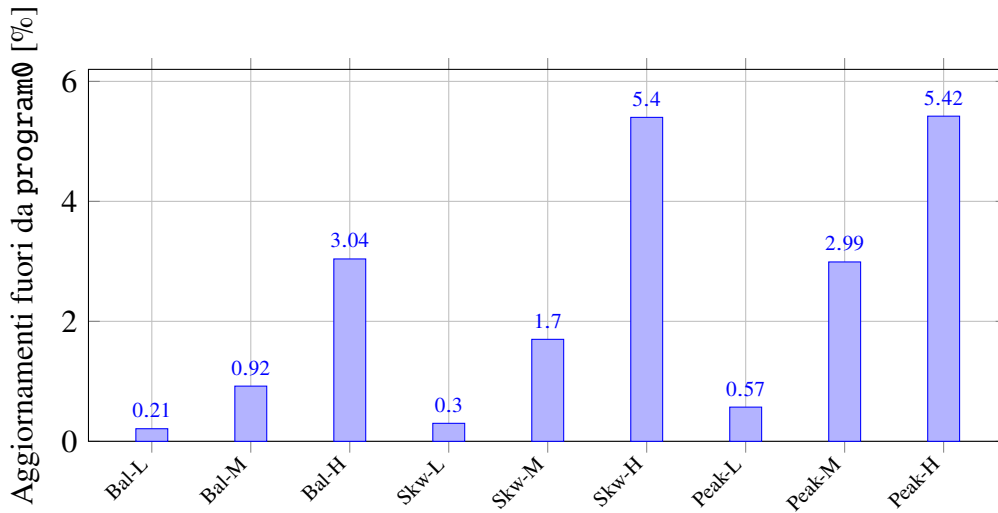


Figura 8.3. Quota di passi-modalità in cui `mp_super` usa una modalità diversa da `program0`.

`program0` occupa fra il 94,58% e il 99,79% dei passi-modalità. La quota adattiva cresce con la domanda e con la concentrazione del traffico: rimane sotto lo 0,6% nei casi *low* e supera il 5% in *skewed high* e *peak high*. Fra i rami specialistici, LTA è il più usato e raggiunge circa il 2,4%; Nmin arriva all'1,25%, mentre fairness, platoon e safe restano residuali. In particolare safe non supera lo 0,76%, coerentemente con il suo ruolo di protezione estrema.

Questi valori mostrano che `mp_super` non sostituisce stabilmente il piano nativo, ma non è equivalente a `fixed_program0`. Opera piuttosto come uno strato correttivo: osserva carico, imbalance, wait imbalance, burstiness, platon e saturazione downstream, quindi interviene localmente per periodi brevi. Il miglioramento rispetto a `fixed_program0` è comunque evidente: otto gruppi su nove migliorano con `mp_super`, e la media delle variazioni percentuali dell'attesa è pari a -30,27%. Anche una quota limitata di decisioni adattive modifica quindi in modo sostanziale l'evoluzione della congestione.

8.6 Discussione dei risultati

La valutazione su Bologna porta a una conclusione diversa da quella ottenuta su Manhattan. Nella griglia sintetica il piano fisso usa durate uniformi e offset nulli, mentre Max-Pressure riduce direttamente i verdi inutili. Sulla rete reale, invece, `fixed_program0` contiene durate e sequenze specifiche dei singoli incroci; sostituirlo continuamente con decisioni locali peggiora le prestazioni osservate.

I risultati sostengono quindi l'ibridazione, non la sostituzione completa del piano nativo. `mp_program0` è semplice e particolarmente competitivo nei casi *skewed* e *peak in high*. `mp_super` vince più gruppi, soprattutto nei regimi *medium* e nei casi *low* non uniformi.

Il risultato più solido è comune ai due controller: il piano nativo rimane la modalità predefinita e Max-Pressure interviene soltanto quando i segnali locali giustificano il costo del cambio.

8.7 Limiti della valutazione

La campagna elimina la censura e usa le stesse popolazioni per tutti gli scenari dello stesso gruppo, producendo un confronto diretto e riproducibile. Le tre repliche permettono di verificare la stabilità dei risultati principali; differenze di pochi secondi fra scenari ibridi vanno invece lette come prestazioni sostanzialmente equivalenti. Bologna è inoltre una sola rete reale e le popolazioni sono generate sinteticamente: *balanced*, *skewed* e *peak* riproducono regimi diversi, ma non derivano da conteggi osservati sul campo.

Le soglie di `mp_super` sono interpretabili, ma sono state calibrate manualmente attraverso campagne sulla stessa mappa; resta quindi un rischio di adattamento eccessivo a Bologna. Infine, i controller mantengono una logica locale e non introducono coordinamento esplicito fra incroci adiacenti. Questi limiti suggeriscono cautela nell'estendere automaticamente le soglie ad altre mappe, ma non riducono l'evidenza centrale della campagna: su Bologna il migliore tra i controller ibridi supera il piano nativo in tutti i gruppi considerati e rende utilizzabile Max-Pressure anche in presenza di un programma semaforico reale già competitivo.

9 Conclusioni

9.1 Sintesi del lavoro

La tesi ha progettato, implementato e valutato una piattaforma per il controllo semaforico adattivo basato su Max-Pressure. Il principio teorico della pressione è stato trasformato in un controller TraCI per SUMO e inserito in una pipeline sperimentale riproducibile, capace di generare popolazioni riutilizzabili tra scenari, eseguire campagne batch e calcolare attesa, tempo di viaggio, time loss, tasso di completamento e censura.

La valutazione è stata articolata in due fasi. Manhattan 8×8 ha fornito un ambiente regolare nel quale isolare il comportamento del Max-Pressure e delle singole estensioni. Bologna ha introdotto una topologia reale, domanda non uniforme e un programma semaforico nativo competitivo. Il passaggio fra le due mappe ha mostrato che l'efficacia di una politica non dipende soltanto dal criterio di pressione, ma anche dal costo degli switch, dalla distribuzione del traffico e dalle temporizzazioni specifiche del piano semaforico disponibile.

9.2 Risultati principali

I risultati principali sono cinque.

1. Su Manhattan, le varianti Max-Pressure battono nettamente `fixed_program0`: il miglior scenario riduce l'attesa del 64,26% in *low*, del 60,09% in *medium* e del 53,05% in *high*, con censura nulla.
2. Le estensioni specialistiche non sono automaticamente vantaggiose. Alcune migliorano in regimi specifici, altre rimangono inattive oppure introducono un costo superiore al beneficio.
3. Su Bologna, il Max-Pressure puro non sostituisce efficacemente il piano nativo. `mp_base` aumenta l'attesa rispetto a `fixed_program0` in tutti i nove gruppi, con differenze comprese fra il 27,37% e il 102,97%.
4. I controller ibridi occupano il primo posto in tutti i gruppi di Bologna. Il miglior ibrido riduce l'attesa rispetto a `fixed_program0` dall'1,19% in *balanced low* fino al 41,47% in *skewed low*; il tempo di viaggio e il percentile 95 migliorano nella stessa direzione.
5. `mp_super` vince cinque gruppi e `mp_program0` quattro. Il super mantiene `program0` per oltre il 94% dei passi-modalità e usa Max-Pressure come intervento locale breve. Il

risultato conferma che l'ibridazione è efficace anche quando il piano statico di partenza è già competitivo.

9.3 Risposta agli obiettivi della tesi

Il primo obiettivo era implementare un controller Max-Pressure funzionante e valutabile. L'obiettivo è stato raggiunto: il controller base, le estensioni e le logiche ibride sono integrati nella stessa interfaccia e producono metriche confrontabili.

Il secondo obiettivo era costruire una metodologia sperimentale riproducibile. Le campagne conservano configurazione risolta, popolazioni, seed, output e report automatici. Il calcolo esplicito della censura ha permesso di individuare campagne iniziali non confrontabili e di progettare la valutazione finale senza cutoff, nella quale tutti i viaggi vengono completati.

Il terzo obiettivo era valutare il comportamento su reti differenti. Manhattan ha mostrato il vantaggio del Max-Pressure in una rete regolare; Bologna ne ha evidenziato i limiti quando esiste un piano statico forte. La risposta complessiva non è quindi una classifica universale: Max-Pressure è efficace, ma nelle reti reali può essere preferibile integrarlo con il piano esistente e attivarlo soltanto nei regimi che giustificano il costo del cambiamento.

9.4 Limiti

La campagna finale Bologna usa tre seed e tutte le simulazioni completano i viaggi senza censura. Questo rende il confronto stabile per le conclusioni principali; differenze di pochi secondi fra controller ibridi vanno invece interpretate come prestazioni sostanzialmente equivalenti. I risultati riguardano inoltre due mappe, una sintetica e una reale, e non possono essere trasferiti automaticamente ad altre topologie.

Le popolazioni di Bologna sono generate dalla pipeline e non da misure di traffico osservate. Le distribuzioni balanced, skewed e peak ampliano i regimi testati, ma rimangono modelli controllati. Le soglie di `mp_super` sono state calibrate manualmente sulla stessa mappa e potrebbero quindi riflettere caratteristiche specifiche di Bologna. Infine, il controller resta locale e non coordina esplicitamente incroci adiacenti.

9.5 Sviluppi futuri

Un primo sviluppo consiste nel validare i controller su ulteriori mappe e su un numero maggiore di seed, separando chiaramente i casi usati per calibrare le soglie da quelli destinati alla valutazione. L'impiego di conteggi di traffico reali permetterebbe inoltre di costruire popolazioni più aderenti alla domanda osservata.

Un secondo sviluppo riguarda la selezione delle modalità. Le soglie del super potrebbero essere ottimizzate automaticamente con grid search o ottimizzazione bayesiana, mantenendo un insieme di validazione indipendente. In alternativa, un decision tree addestrato sui log potrebbe apprendere regole interpretabili da confrontare con quelle manuali.

Un terzo sviluppo è l'introduzione di forme leggere di coordinamento fra incroci adiacenti o di indicatori regionali, coerentemente con approcci recenti come Smoothing-MP e N-MP [XBL24; LG24]. Infine, il meccanismo di spillback potrebbe essere reso meno rigido, penalizzando i singoli movimenti invece di escludere un'intera fase quando una sola uscita è satura.

Nel complesso, il lavoro mostra che il valore del Max-Pressure non sta necessariamente nella sostituzione completa del piano semaforico esistente, ma nella capacità di affiancarlo con decisioni locali mirate, osservabili e attivate solo quando il traffico lo richiede.

A Configurazioni sperimentali

Questa appendice raccoglie il materiale tecnico necessario per replicare gli esperimenti, senza interrompere la lettura dei capitoli principali. Le configurazioni complete, i log e le tabelle generate automaticamente rimangono nel repository del progetto. Nel seguito, i percorsi indicati sono relativi alla radice di tale repository.

A.1 Configurazioni delle campagne

La campagna Manhattan usa un sottoinsieme rappresentativo: `mp_base`, LTA 0,60, Nmin 0,80/2, downstream 0,4, spillback 90/80, fairness 5/20, platoon x4, margine relativo 0,05 e `fixed_program0`. La campagna Bologna finale usa invece `bologna_thesis_v1`: `mp_base`, `mp_program0`, `mp_super`, LTA 0,60, Nmin forte 1,20/4, spillback 90/80, fairness 5/20 e platoon x2, più `fixed_program0` e `fixed_tuned`.

Tabella A.1. Sintesi delle campagne sperimentali principali.

Campagna	Configurazione
Manhattan	Mappa manhattan8x8_100pc; domande <i>low</i> , <i>medium</i> e <i>high</i> ; 5 seed; nessun cutoff anticipato; baseline <code>fixed_program0</code> ; scenario pack <code>tuning_matrix_v2</code> .
Bologna	Mappa bologna; domande <i>low</i> , <i>medium</i> e <i>high</i> ; popolazioni balanced, skewed e peak; 3 seed nella campagna finale; nessun cutoff anticipato; baseline <code>fixed_program0</code> ; scenario pack <code>bologna_thesis_v1</code> .

Le soglie principali di `mp_super` sono: riferimento di carico 3,5, ingresso nei rami Max-Pressure a 0,38, ritorno a `program0` sotto 0,28, regime *high* a 0,62, ramo Nmin forte a 0,72, imbalance e wait imbalance a 0,50, burstiness a 0,10, safe downstream a 0,97 e conferma della modalità per quattro aggiornamenti consecutivi.

A.2 Output e risultati completi

Le tabelle complete non sono riportate integralmente nel testo per non appesantire la tesi. Per ogni campagna, la pipeline genera automaticamente i file `table.txt`, `summary_by_group.csv`, `summary_vs_base.csv` e `winners.txt`. I risultati Manhattan usati nel Capitolo 6 si trovano in `logs/ablation/runs/run_0028_20260612_131724`. I

risultati Bologna usati nel Capitolo 8 provengono dalla campagna finale `logs/ablation/runs/run_0029_20260614_143342`.

A.3 Struttura essenziale del software

Il repository è organizzato intorno a pochi file principali. `runner.py` esegue una singola simulazione SUMO, costruisce il controller scelto e raccoglie le metriche. `src/controllers/max_pressure.py` contiene Max-Pressure, le estensioni specialistiche, `mp_program0` e `mp_super`. `generate_population.py` genera le popolazioni, incluse le distribuzioni `balanced`, `skewed` e `peak`. `utils/run_ablation.py` definisce gli scenario pack, esegue le campagne batch e produce i report. Infine, la cartella `tests/` contiene i test automatici usati per controllare scenari, popolazioni e logiche del meta-controller.

B Dettagli implementativi

Questa appendice riporta alcuni estratti rappresentativi del codice reale del progetto. Non si tratta di pseudocodice: i nomi delle funzioni, delle variabili e delle strutture corrispondono all'implementazione in `src/controllers/max_pressure.py`. Per contenere la lunghezza, tuttavia, i listati sono stati adattati eliminando parti accessorie come annotazioni di tipo complete, cache interne e controlli ripetitivi. L'obiettivo è mostrare il nucleo dei metodi senza trasformare l'appendice nella copia integrale dei file sorgente.

Prima dei listati è utile chiarire le principali funzioni e strutture richiamate:

- `traci` è l'interfaccia Python con cui il controller osserva e comanda la simulazione SUMO.
- `getControlledLinks()` restituisce, per ogni posizione della stringa semaforica, i collegamenti controllati tra corsia in ingresso e corsia in uscita.
- `tl_data` è la struttura interna associata a un semaforo e contiene, tra le altre informazioni, le fasi principali e i movimenti abilitati da ciascuna fase.
- `_is_downstream_blocked()` verifica se una corsia di uscita è considerata satura secondo la logica anti-spillback.
- `_movement_downstream_penalty()` calcola la penalità da sottrarre alla pressione quando una corsia downstream è congestionata.
- `snapshot` raccoglie gli indicatori osservati online da `mp_super`: carico, imbalance, wait imbalance, burstiness, plotoni e saturazione downstream.

B.1 Costruzione dei movimenti dalle fasi SUMO

Il primo estratto mostra come una fase SUMO venga trasformata nell'insieme dei movimenti controllati. Il codice considera principali solo le fasi con almeno un verde e senza giallo; per ogni posizione verde della stringa semaforica recupera da TraCI le corsie di ingresso e uscita corrispondenti.

```
1 def _build_traffic_light_data(self, tl_id: str, logic):
2     controlled_links = traci.trafficlight.getControlledLinks(tl_id)
3     movements_by_phase = {}
4
5     for phase_index, phase in enumerate(logic.phases):
6         if not self._is_main_phase_state(phase.state):
7             continue
```

```

8
9     seen_movements = set()
10    movements = []
11
12    for signal_index, signal_state in enumerate(phase.state):
13        if signal_state not in ("g", "G"):
14            continue
15        if signal_index >= len(controlled_links):
16            continue
17
18        for in_lane, out_lane, _ in controlled_links[
19            signal_index]:
20            movement = (in_lane, out_lane)
21            if movement in seen_movements:
22                continue
23            seen_movements.add(movement)
24            movements.append(movement)
25
26    if movements:
27        movements_by_phase[phase_index] = movements
28
29    return movements_by_phase
30
31 def _is_main_phase_state(state: str) -> bool:
32     return (
33         any(char in ("g", "G") for char in state)
34         and not any(char in ("y", "Y") for char in state)
35     )

```

Listato B.1. Estrazione dei movimenti controllati dalle fasi SUMO.

B.2 Calcolo della pressione di fase

Il secondo estratto contiene il nucleo del Max-Pressure: per ogni fase vengono sommate le differenze tra coda in ingresso e coda in uscita dei movimenti abilitati. La stessa funzione integra anche la penalità downstream e il blocco anti-spillback, quando abilitati.

```

1 def _phase_pressures(self, tl_data):
2     pressures = {}
3     phase_demand = {}

```



```

4
5  def queue_on_lane(lane_id):
6      return float(traci.lane.getLastStepHaltingNumber(lane_id))
7
8  for phase_index, movements in tl_data.movements_by_phase.items():
9      pressure = 0.0
10     demand_sum = 0.0
11     phase_blocked = False
12
13     for in_lane, out_lane in movements:
14         if self._is_downstream_blocked(out_lane):
15             phase_blocked = True
16             break
17
18         in_queue = queue_on_lane(in_lane)
19         out_queue = queue_on_lane(out_lane)
20         downstream_penalty = self._movement_downstream_penalty(
21             out_lane)
22
23         demand_sum += in_queue
24         pressure += in_queue - out_queue - downstream_penalty
25
26     if phase_blocked:
27         pressures[phase_index] = float("-inf")
28         phase_demand[phase_index] = 0.0
29     else:
30         pressures[phase_index] = pressure
31         phase_demand[phase_index] = demand_sum
32
33     return pressures, phase_demand

```

Listato B.2. Calcolo della pressione associata alle fasi principali.

B.3 Selezione delle modalità di mp_super

Il terzo estratto mostra la logica rule-based del meta-controller. La decisione dipende da indicatori calcolati online: carico, sbilanciamento della domanda, sbilanciamento dell'attesa,

burstiness, platon e saturazione downstream. La modalità `program0` rimane il comportamento predefinito quando il carico non supera le soglie.

```
1 def _select_super_mode(self, snapshot, current_mode="program0"):
2     bursty = snapshot.burstiness >= self.super_burstiness_threshold
3     wait_starved = snapshot.wait_imbalance >= self.
4         super_wait_imbalance_threshold
5     imbalanced = snapshot.imbalance >= self.
6         super_imbalance_threshold
7     platoon = snapshot.platoon_score >= self.
8         super_platoon_threshold
9
10    if (
11        self.super_safe_enabled
12        and snapshot.downstream_score >= self.
13            super_downstream_threshold
14        and snapshot.load >= self.super_nmin_load
15        and (bursty or wait_starved)
16    ):
17        return "safe"
18
19    program0_threshold = (
20        self.super_low_load
21        if current_mode == "program0"
22        else self.super_program0_exit_load
23    )
24    if snapshot.load < program0_threshold:
25        return "program0"
26
27    if snapshot.load >= self.super_nmin_load:
28        if bursty:
29            return "nmin"
30        if wait_starved and imbalanced:
31            return "lta"
32        if wait_starved and not imbalanced:
33            return "fair"
34        if platoon and not bursty:
35            return "platoon"
36        return "nmin"
37
38    if snapshot.load >= self.super_high_load:
```

```

35     if bursty:
36         return "nmin"
37     if wait_starved and imbalanced:
38         return "lta"
39     if wait_starved and not bursty and not imbalanced:
40         return "fair"
41     if platoon and not wait_starved:
42         return "platoon"
43     return "base"
44
45     if wait_starved and imbalanced and not bursty:
46         return "lta"
47     if wait_starved and not imbalanced and not bursty:
48         return "fair"
49     if platoon and not wait_starved:
50         return "platoon"
51     if bursty:
52         return "base"
53     return "base"

```

Listato B.3. Regole di selezione della modalità in mp_super.

Riferimenti bibliografici

- [And+18] Leah Anderson et al. «Stability and Implementation of a Cycle-Based Max Pressure Controller for Signalized Traffic Networks». In: *Networks and Heterogeneous Media* 13.2 (2018), pp. 241–260. DOI: 10.3934/nhm.2018011.
- [CF12] Burak Cesme e Peter G. Furth. «Multiheadway Gap-Out Logic for Actuated Control on Multilane Approaches». In: *Transportation Research Record* 2311.1 (2012), pp. 117–123. DOI: 10.3141/2311-11.
- [Ecl26a] Eclipse SUMO Project. *SUMO Documentation*. 2026. URL: <https://sumo.dlr.de/docs/> (visitato il 13/06/2026).
- [Ecl26b] Eclipse SUMO Project. *TraCI Documentation*. 2026. URL: <https://sumo.dlr.de/docs/TraCI/index.html> (visitato il 13/06/2026).
- [Gar83] Nathan H. Gartner. «OPAC: A Demand-Responsive Strategy for Traffic Signal Control». In: *Transportation Research Record* 906 (1983), pp. 75–81.
- [Gre+14] Jean Gregoire et al. «Back-Pressure Traffic Signal Control with Unknown Routing Rates». In: *Proceedings of the 19th IFAC World Congress*. 2014, pp. 11332–11337. URL: <https://arxiv.org/abs/1401.3357> (visitato il 17/06/2026).
- [Gre+15] Jean Gregoire et al. «Capacity-Aware Backpressure Traffic Signal Control». In: *IEEE Transactions on Control of Network Systems* 2.2 (2015), pp. 164–173. DOI: 10.1109/TCNS.2014.2378871.
- [Kou+14] Anastasios Kouvelas et al. «Maximum Pressure Controller for Stabilizing Queues in Signalized Arterial Networks». In: *Transportation Research Record* 2421.1 (2014), pp. 133–141. DOI: 10.3141/2421-15.
- [Le+15] Tung Le et al. «Decentralized Signal Control for Urban Road Networks». In: *Transportation Research Part C: Emerging Technologies* 58 (2015), pp. 431–450. DOI: 10.1016/j.trc.2014.11.009.
- [Lev23] Michael W. Levin. «Max-Pressure Traffic Signal Timing: A Summary of Methodological and Experimental Results». In: *Journal of Transportation Engineering, Part A: Systems* 149.4 (2023), p. 03123001. DOI: 10.1061/JTEPBS.TEENG-7578.
- [LHO20] Michael W. Levin, Jeffrey Hu e Michael Odell. «Max-Pressure Signal Control with Cyclical Phase Structure». In: *Transportation Research Part C: Emerging Technologies* 120 (2020), p. 102828. DOI: 10.1016/j.trc.2020.102828.

- [LJ19] Li Li e Saif Eddin Jabari. «Position Weighted Backpressure Intersection Control for Urban Networks». In: *Transportation Research Part B: Methodological* 128 (2019), pp. 435–461. doi: 10.1016/j.trb.2019.08.005.
- [LG22] Hao Liu e Vikash V. Gayah. «A Novel Max Pressure Algorithm Based on Traffic Delay». In: *Transportation Research Part C: Emerging Technologies* 143 (2022), p. 103803. doi: 10.1016/j.trc.2022.103803.
- [LG24] Hao Liu e Vikash V. Gayah. «N-MP: A Network-State-Based Max Pressure Algorithm Incorporating Regional Perimeter Control». In: *Transportation Research Part C: Emerging Technologies* 168 (2024), p. 104725. doi: 10.1016/j.trc.2024.104725.
- [Lop+18] Pablo Alvarez Lopez et al. «Microscopic Traffic Simulation using SUMO». In: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. 2018, pp. 2575–2582. doi: 10.1109/ITSC.2018.8569938.
- [MUH20] Pablo Mercader, Wajdi Uwayid e Jack Haddad. «Max-Pressure Traffic Controller Based on Travel Times: An Experimental Analysis». In: *Transportation Research Part C: Emerging Technologies* 110 (2020), pp. 275–290. doi: 10.1016/j.trc.2019.10.002.
- [Noa+21] Mohammad Noaeen et al. «Real-Time Decentralized Traffic Signal Control for Congested Urban Networks Considering Queue Spillbacks». In: *Transportation Research Part C: Emerging Technologies* 133 (2021), p. 103407. doi: 10.1016/j.trc.2021.103407.
- [RB91] Dennis I. Robertson e Robert D. Bretherton. «Optimizing Networks of Traffic Signals in Real Time: The SCOOT Method». In: *IEEE Transactions on Vehicular Technology* 40.1 (1991), pp. 11–15.
- [SY18] Xiaotong Sun e Yafeng Yin. «A Simulation Study on Max Pressure Control of Signalized Intersections». In: *Transportation Research Record* 2672.18 (2018), pp. 117–127. doi: 10.1177/0361198118786840.
- [TE92] Leandros Tassiulas e Anthony Ephremides. «Stability Properties of Constrained Queueing Systems and Scheduling Policies for Maximum Throughput in Multihop Radio Networks». In: *IEEE Transactions on Automatic Control* 37.12 (1992), pp. 1936–1948. doi: 10.1109/9.182479.
- [Var13] Pravin Varaiya. «Max Pressure Control of a Network of Signalized Intersections». In: *Transportation Research Part C: Emerging Technologies* 36 (2013), pp. 177–195. doi: 10.1016/j.trc.2013.08.014.

- [Wan+22] Xingmin Wang et al. «Learning the Max Pressure Control for Urban Traffic Networks Considering the Phase Switching Loss». In: *Transportation Research Part C: Emerging Technologies* 140 (2022), p. 103670. doi: 10.1016/j.trc.2022.103670.
- [Weg+08] Axel Wegener et al. «TraCI: An Interface for Coupling Road Traffic and Network Simulators». In: *Proceedings of the 11th Communications and Networking Simulation Symposium*. 2008, pp. 155–163. doi: 10.1145/1400713.1400740.
- [Won+12] Tichakorn Wongpiromsarn et al. «Distributed Traffic Signal Control for Maximum Network Throughput». In: *Proceedings of the 15th International IEEE Conference on Intelligent Transportation Systems*. 2012, pp. 588–595. doi: 10.1109/ITSC.2012.6338817.
- [Wu+18] Jian Wu et al. «Delay-Based Traffic Signal Control for Throughput Optimality and Fairness at an Isolated Intersection». In: *IEEE Transactions on Vehicular Technology* 67.2 (2018), pp. 896–909. doi: 10.1109/TVT.2017.2760820.
- [XBL24] Te Xu, Simanta Barman e Michael W. Levin. «Smoothing-MP: A Novel Max-Pressure Signal Control Considering Signal Coordination to Smooth Traffic in Urban Networks». In: *Transportation Research Part C: Emerging Technologies* 166 (2024), p. 104760. doi: 10.1016/j.trc.2024.104760.